

Lecture 18:

3D Vision

Slides credit: Justin Johnson

Recall: 2D Detection and Segmentation

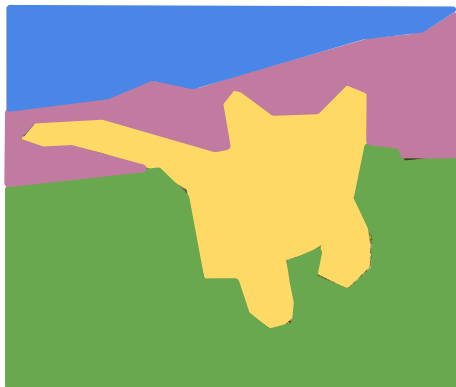
Classification



CAT

No spatial extent

Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation



DOG, DOG, CAT

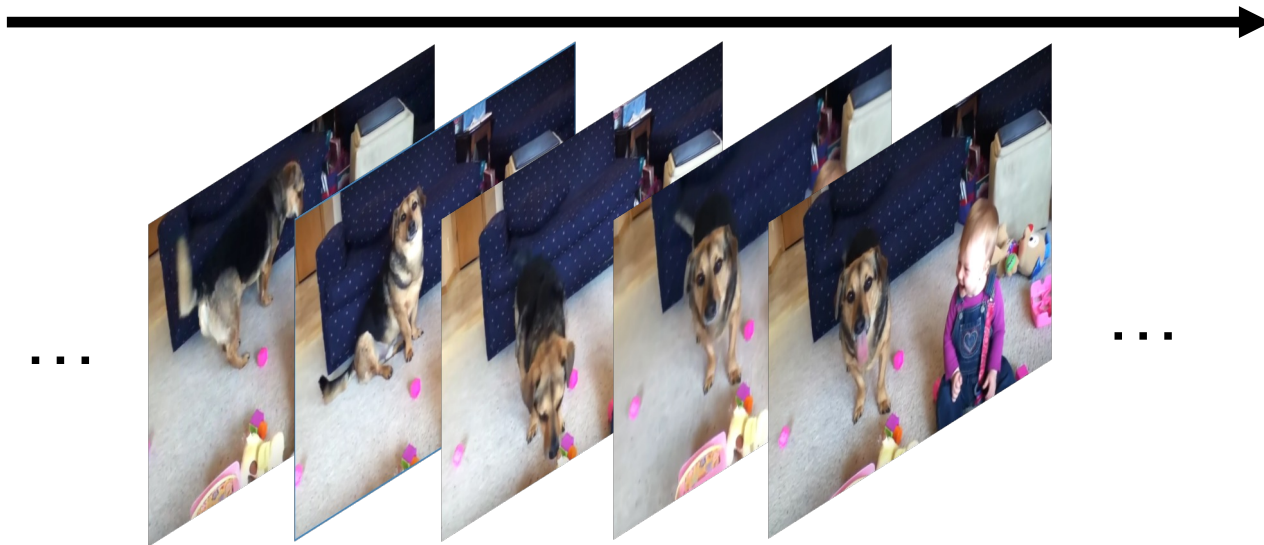
[This image is CC0 public domain](#)

Recall: Video = 2D + Time

A video is a **sequence** of images

4D tensor: $T \times 3 \times H \times W$

(or $3 \times T \times H \times W$)



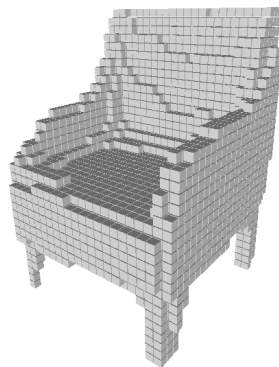
[This image](#) is [CC0 public domain](#)

Focus on Two Problems today

Predicting 3D
Shapes from single
image

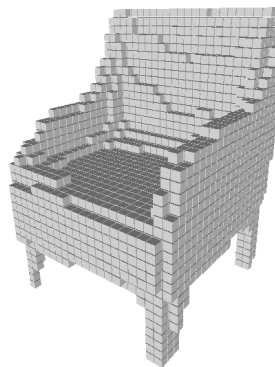


Input Image



3D Shape

Processing 3D
input data



3D Shape



Chair

Many more topics in 3D Vision!

3D Representations

Computing Correspondences

Multi-view Stereo

Structure from Motion

3D Object Pose Estimation

Simultaneous Localization and Mapping (SLAM)

Differentiable Graphics

3D Sensors

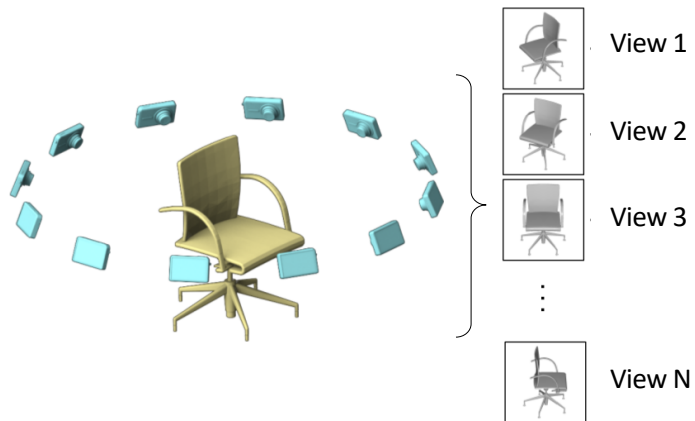
.....

Multi-View CNN



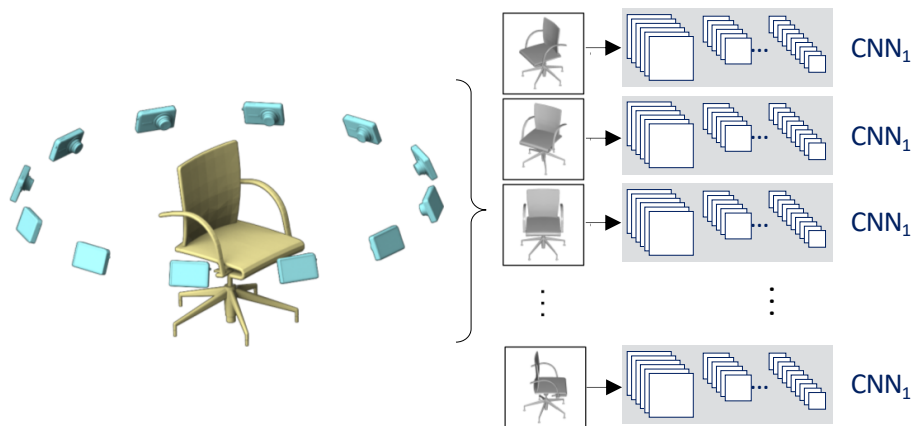
Su et al. ICCV 2015

Multi-View CNN



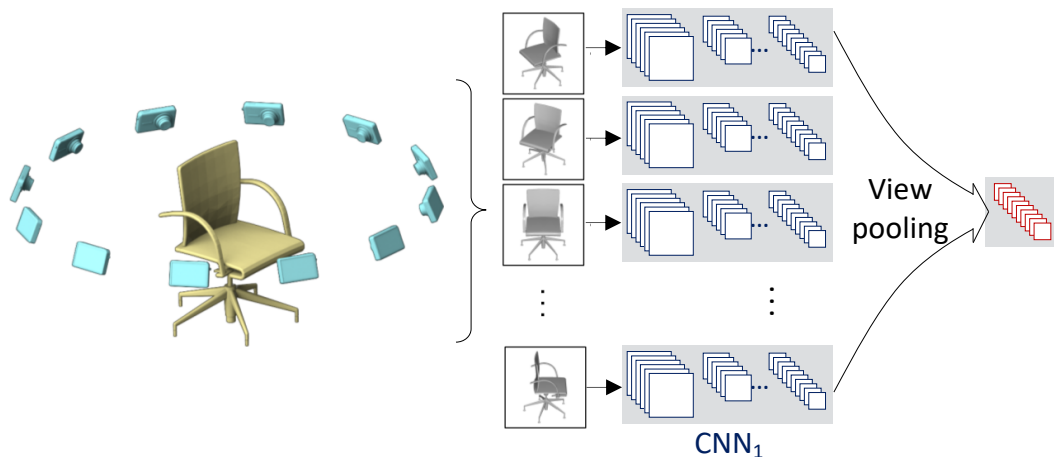
Su et al. ICCV 2015

Multi-View CNN



CNN₁: a ConvNet extracting image features

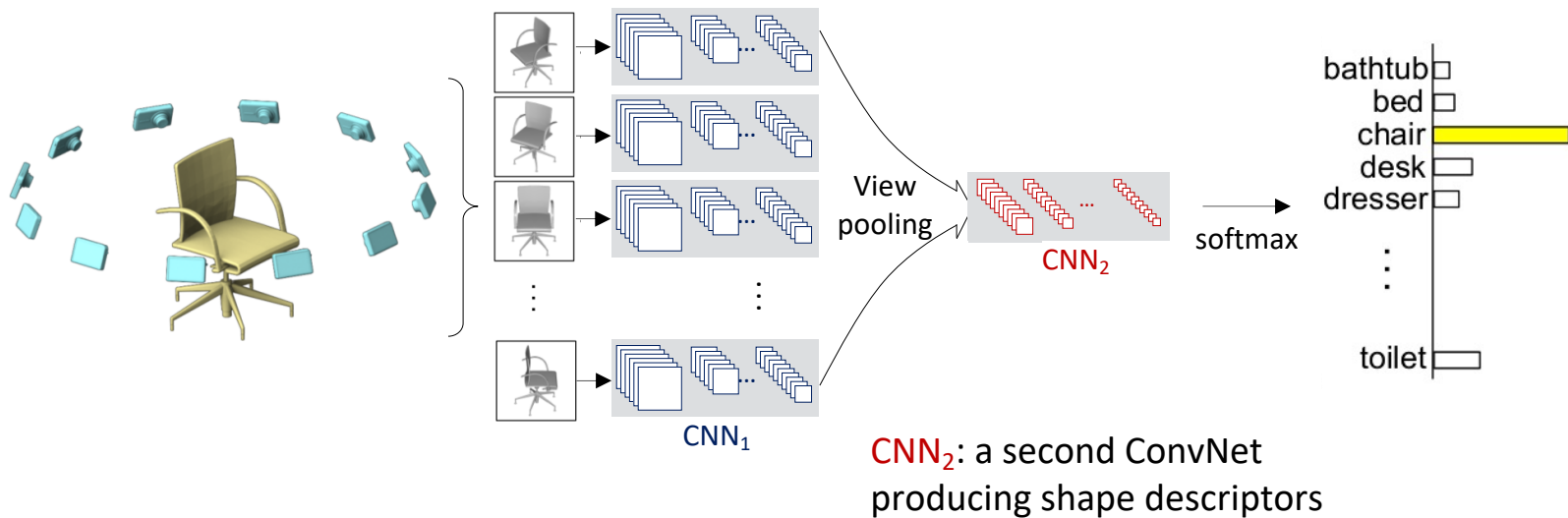
Multi-View CNN



View pooling: element-wise
max-pooling across all views

Su et al. ICCV 2015

Multi-View CNN



Su et al. ICCV 2015

Experiments – Classification & Retrieval

		Method	Classification (Accuracy)	Retrieval (mAP)
Non-deep {		SPH	68.2%	33.3%
		LFD	75.5%	40.9%
		3D ShapeNets	77.3%	49.2%
		FV, 12 views	84.8%	43.9%
		CNN, 12 views	88.6%	62.8%
		MVCNN, 12 views	89.9%	70.1%
		MVCNN+metric, 12 views	89.5%	80.2%
		MVCNN, 80 views	90.1%	70.4%
		MVCNN+metric, 80 views	90.1%	79.5%

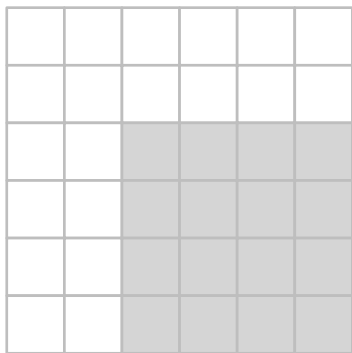
On ModelNet 40

Su et al. ICCV 2015

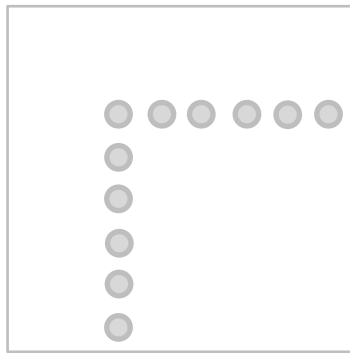
3D Shape Representations



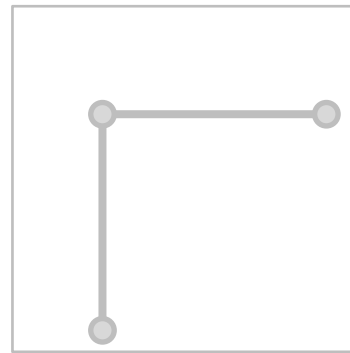
Depth
Map



Voxel
Grid



Pointcloud



Mesh



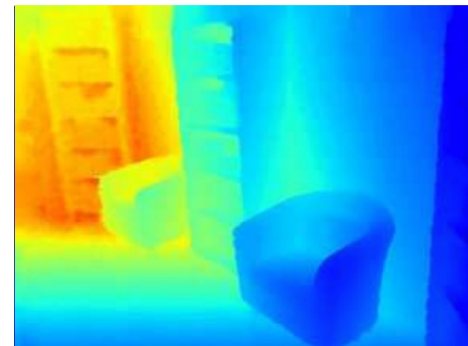
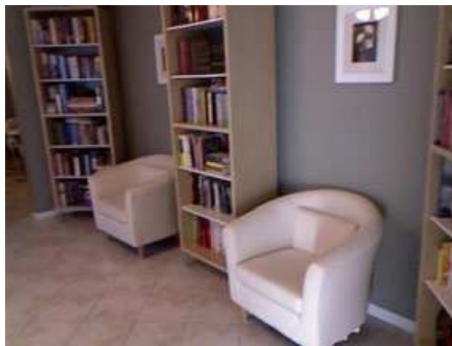
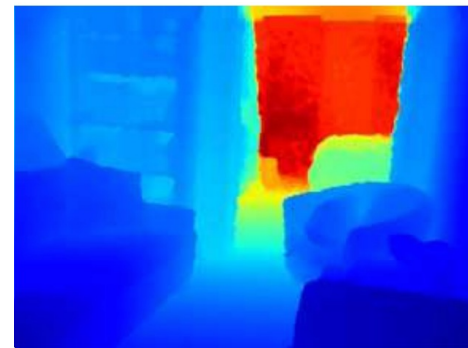
Implicit
Surface

3D Shape Representations: Depth Map

For each pixel, **depth map** gives distance from the camera to the object in the world at that pixel

RGB image + Depth image
= RGB-D Image (2.5D)

This type of data can be recorded directly for some types of 3D sensors (e.g. Intel Realsense, Microsoft Kinect)



RGB Image: $3 \times H \times W$ Depth Map: $H \times W$

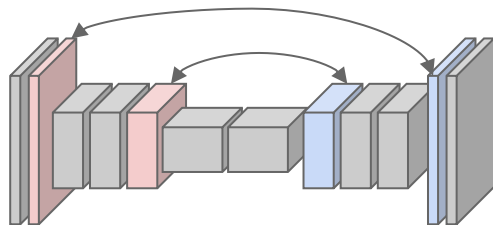
Eigen and Fergus, "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture", ICCV 2015

Predicting Depth Maps

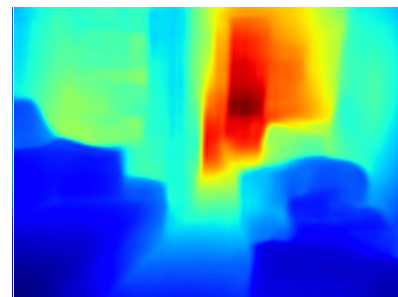
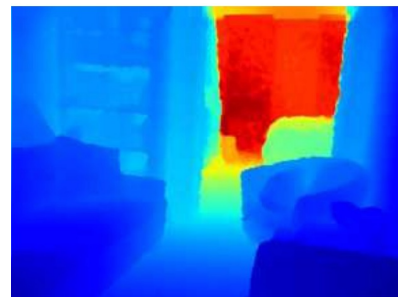
Predicted Depth Image:
 $1 \times H \times W$



RGB Input Image:
 $3 \times H \times W$



**Fully Convolutional
network**

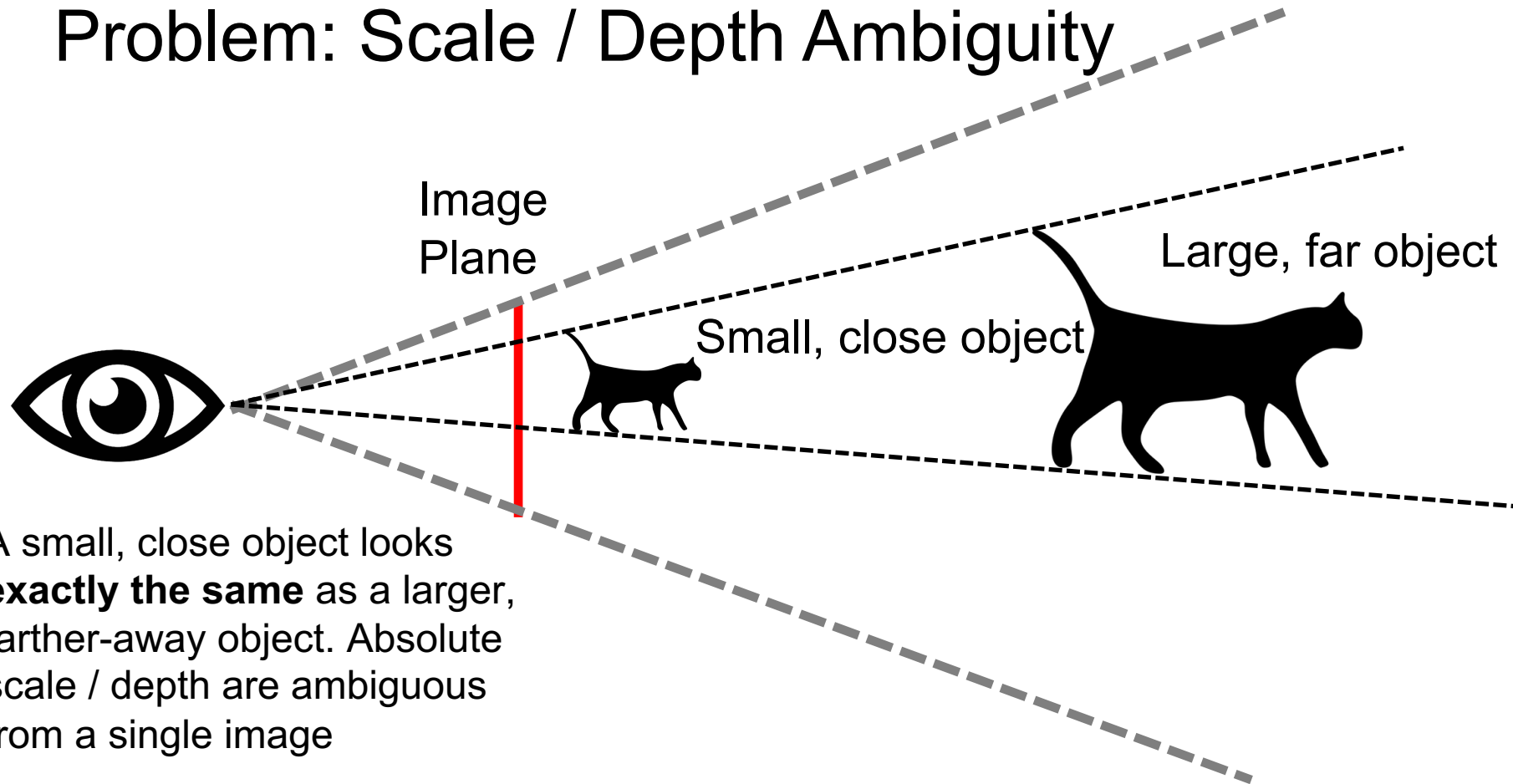


Predicted Depth Image:
 $1 \times H \times W$

**Per-Pixel Loss
(L2 Distance)**

Eigen, Puhrsh, and Fergus, "Depth Map Prediction from a Single Image using a Multi-Scale Deep Network", NeurIPS 2014
Eigen and Fergus, "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture", ICCV 2015

Problem: Scale / Depth Ambiguity

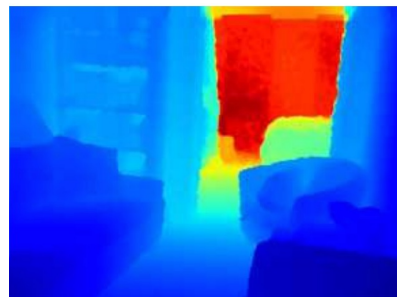


Predicting Depth Maps

Predicted Depth Image:
1 x H x W

Scale invariant loss

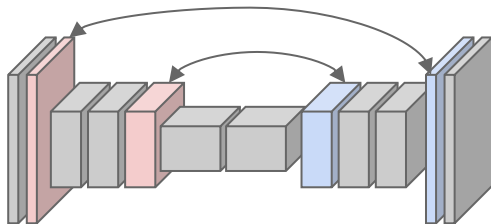
$$\begin{aligned} D(y, y^*) &= \frac{1}{2n^2} \sum_{i,j} ((\log y_i - \log y_j) - (\log y_i^* - \log y_j^*))^2 \\ &= \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \sum_{i,j} d_i d_j = \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \left(\sum_i d_i \right)^2 \end{aligned}$$



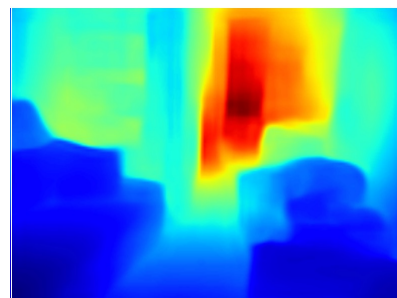
Per-Pixel Loss
(Scale invariant)



RGB Input Image:
3 x H x W



Fully Convolutional network

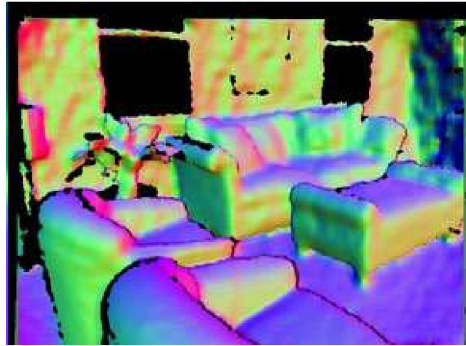
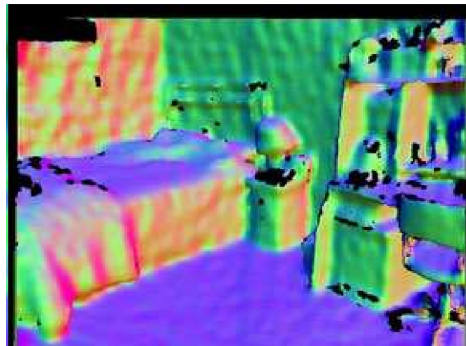


Predicted Depth Image:
1 x H x W

Eigen, Puhrsh, and Fergus, "Depth Map Prediction from a Single Image using a Multi-Scale Deep Network", NeurIPS 2014
Eigen and Fergus, "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture", ICCV 2015

3D Shape Representations: Surface Normals

For each pixel, **surface normals** give a vector giving the normal vector to the object in the world for that pixel



RGB Image: $3 \times H \times W$ Normals: $3 \times H \times W$

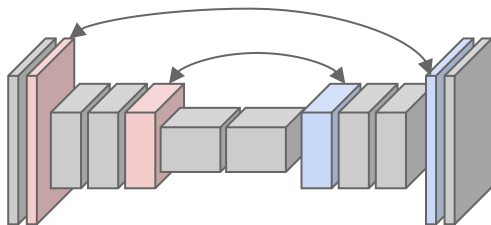
Eigen and Fergus, "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture", ICCV 2015

Predicting Normals

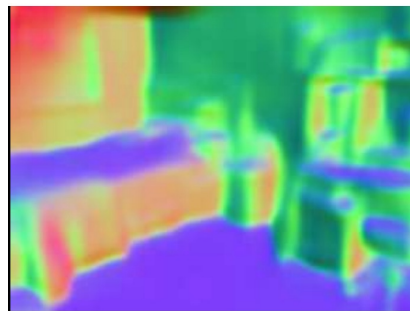
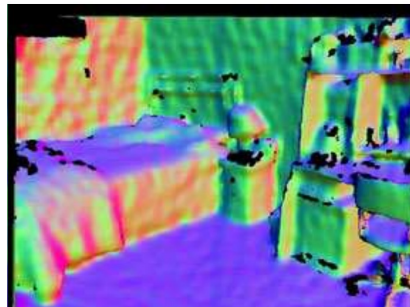
Ground-truth Normals:
 $3 \times H \times W$



RGB Input Image:
 $3 \times H \times W$



Fully Convolutional network



Predicted Normals:
 $3 \times H \times W$

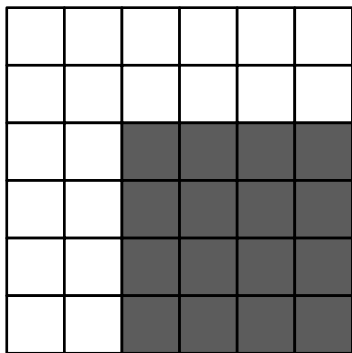
Per-Pixel Loss:
 $(x \cdot y) / (|x||y|)$

Recall:
 $x \cdot y$
 $= |x| |y| \cos \theta$

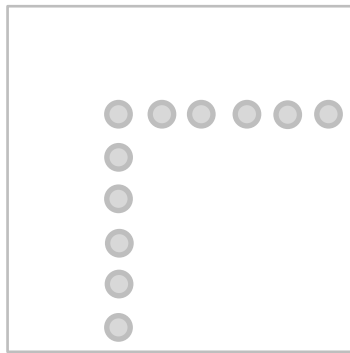
3D Shape Representations



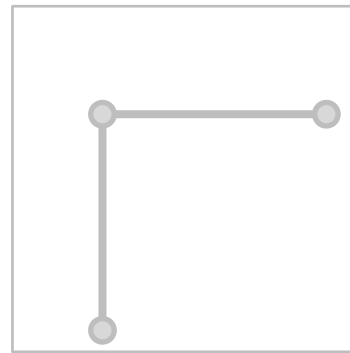
Depth
Map



Voxel
Grid



Pointcloud



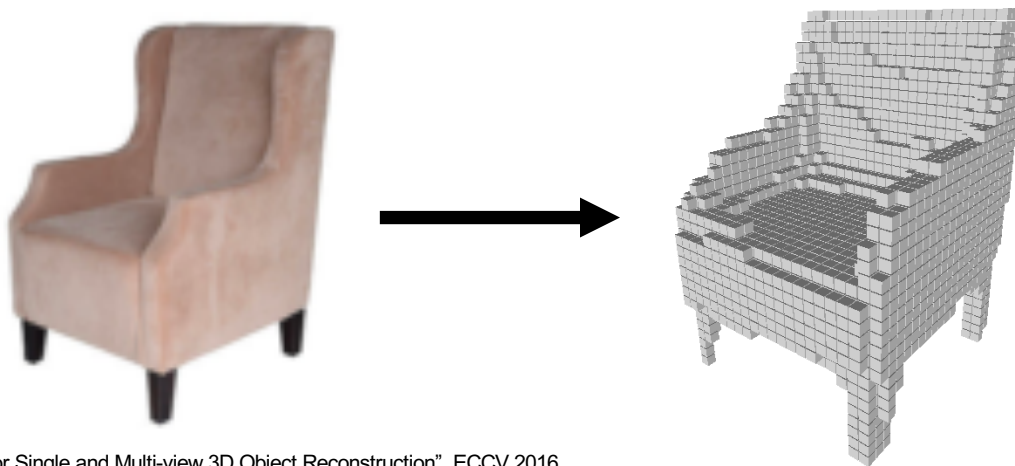
Mesh



Implicit
Surface

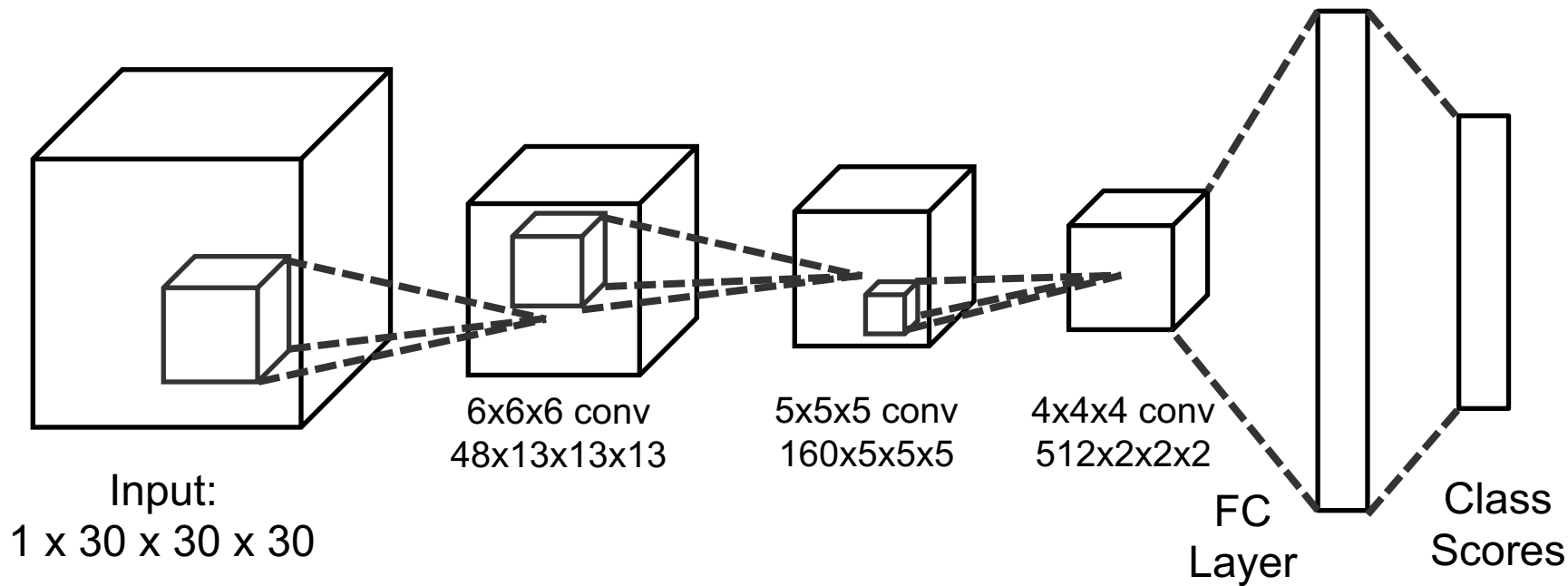
3D Shape Representations: Voxels

- Represent a shape with a $V \times V \times V$ grid of occupancies
- Just like segmentation masks in Mask R-CNN, but in 3D!
- (+) Conceptually simple: just a 3D grid!
- (-) Need high spatial resolution to capture fine structures
- (-) Scaling to high resolutions is nontrivial!



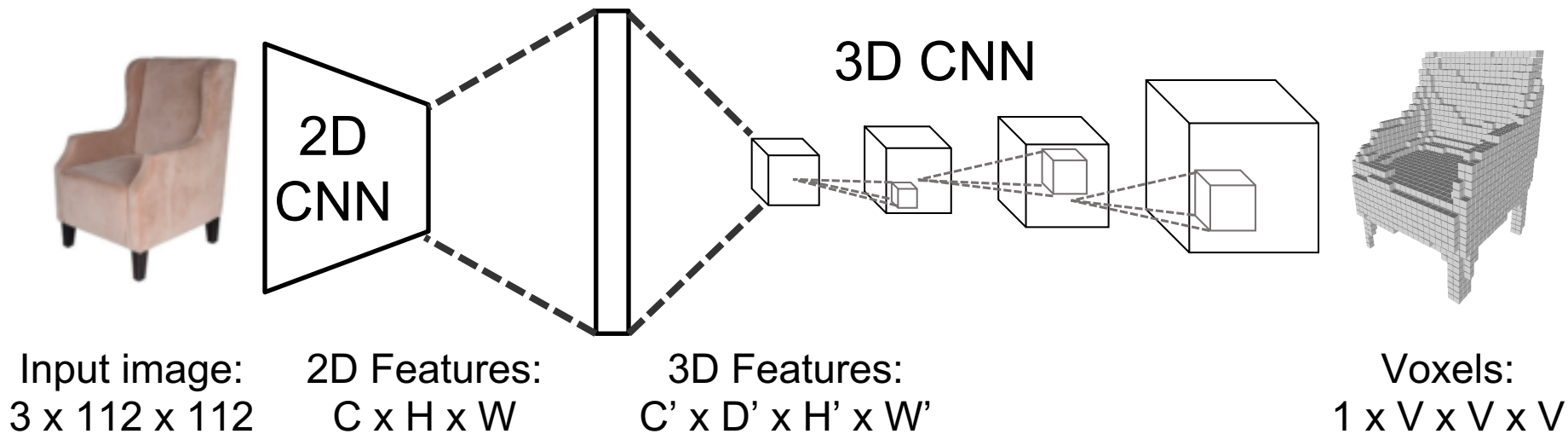
Choy et al, "3D-R2N2: A Unified Approach for Single and Multi-view 3D Object Reconstruction", ECCV 2016

Processing Voxel Inputs: 3D Convolution



Train with classification loss

Generating Voxel Shapes: 3D Convolution



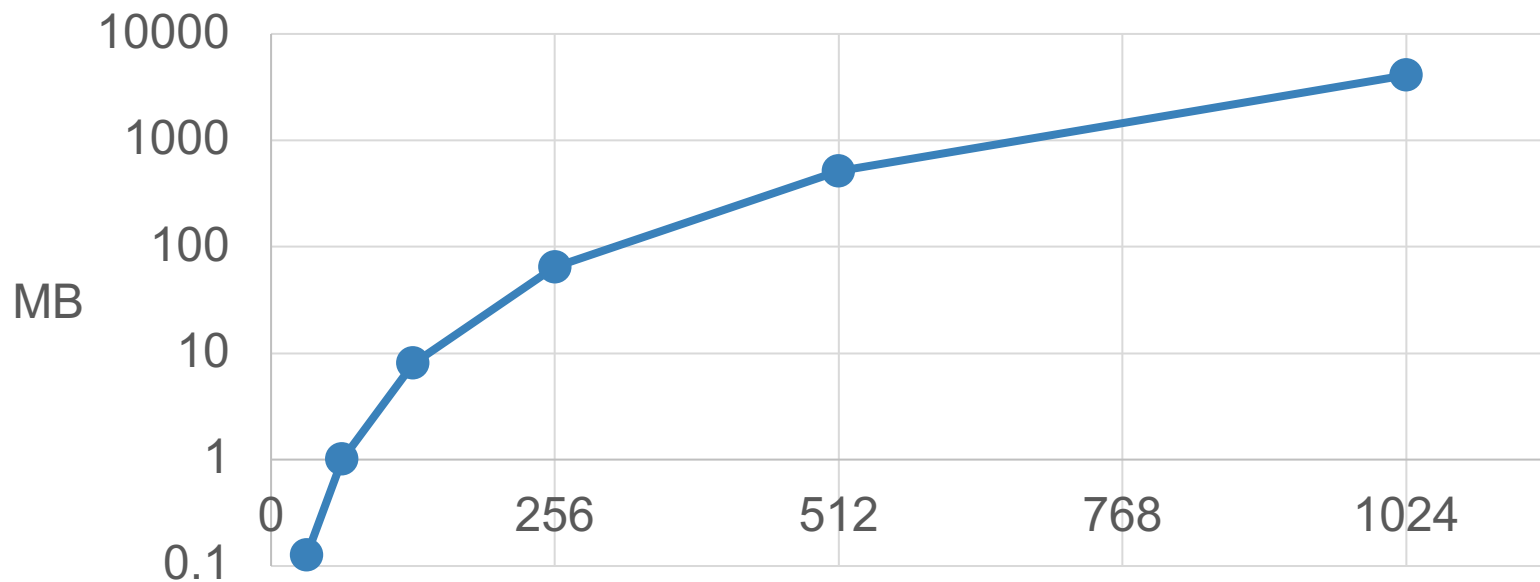
Train with per-voxel cross-entropy loss

Choy et al, "3D-R2N2: A Unified Approach for Single and Multi-view 3D Object Reconstruction", ECCV 2016

Voxel Problems: Memory Usage

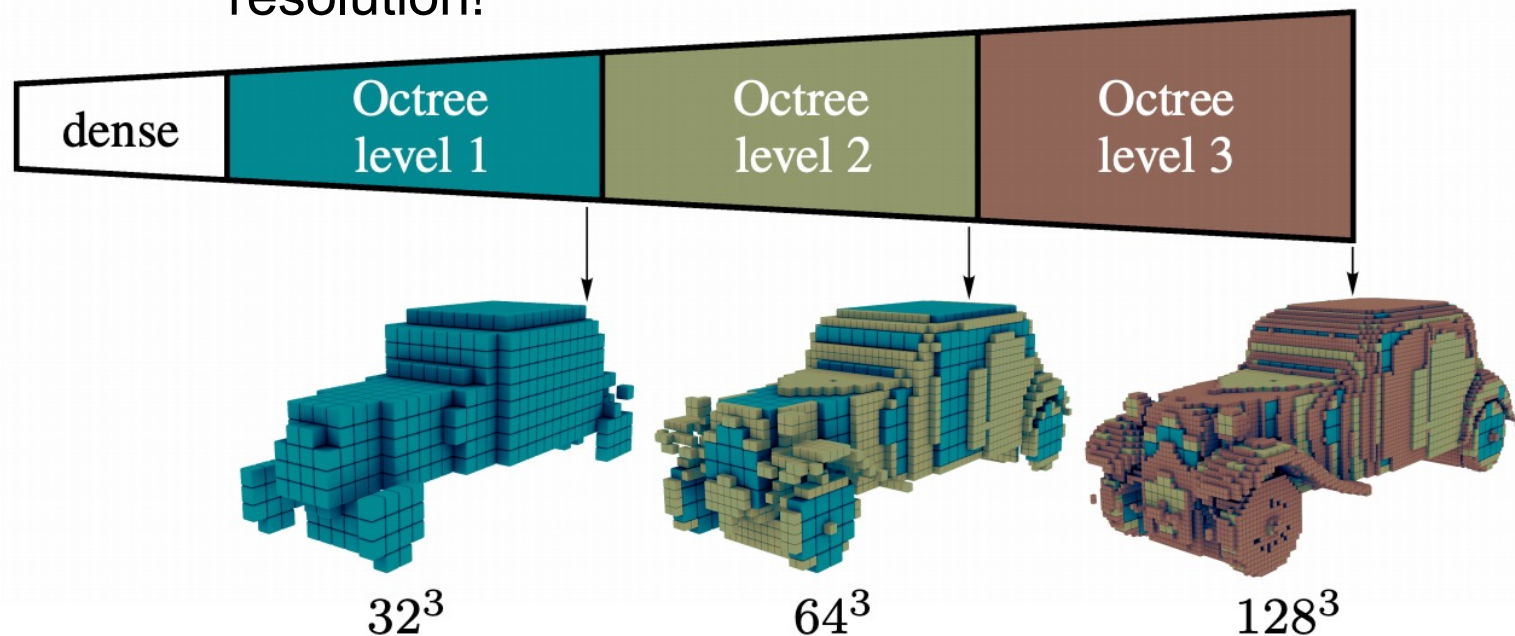
Storing 1024^3 voxel grid takes 4GB of memory!

Voxel memory usage ($V \times V \times V$ float32 numbers)



Scaling Voxels: Oct-Trees

Use voxel grids with heterogenous resolution!

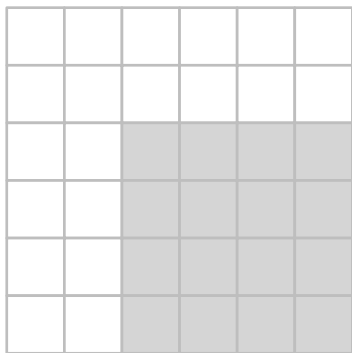


Tatarchenko et al, "Octree Generating Networks: Efficient Convolutional Architectures for High-resolution 3D Outputs", ICCV 2017

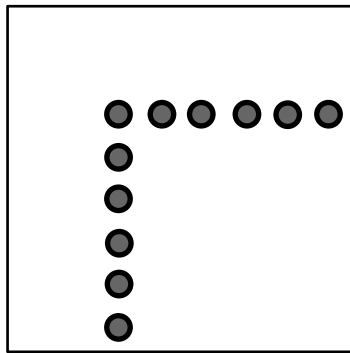
3D Shape Representations



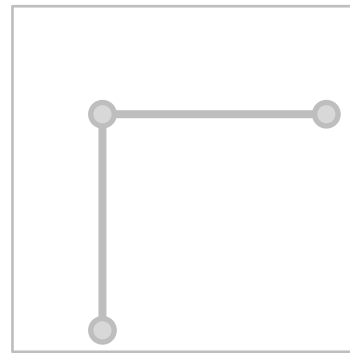
Depth
Map



Voxel
Grid



Pointcloud



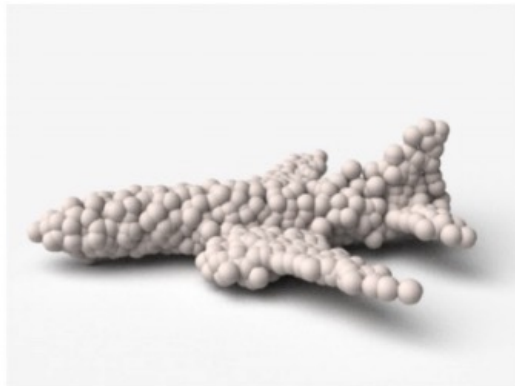
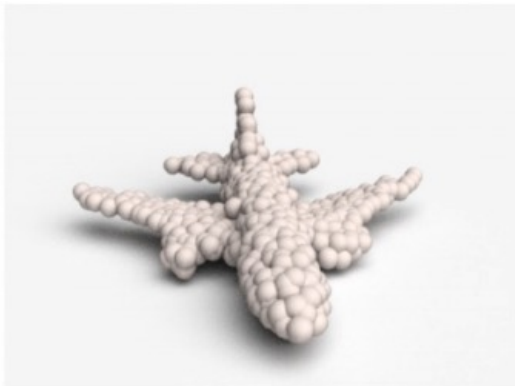
Mesh



Implicit
Surface

3D Shape Representations: Point Cloud

- Represent shape as a set of P points in 3D space
- (+) Can represent fine structures without huge numbers of points
- () Requires new architecture, losses, etc
- (-) Doesn't explicitly represent the surface of the shape: extracting a mesh for rendering or other applications requires post-processing



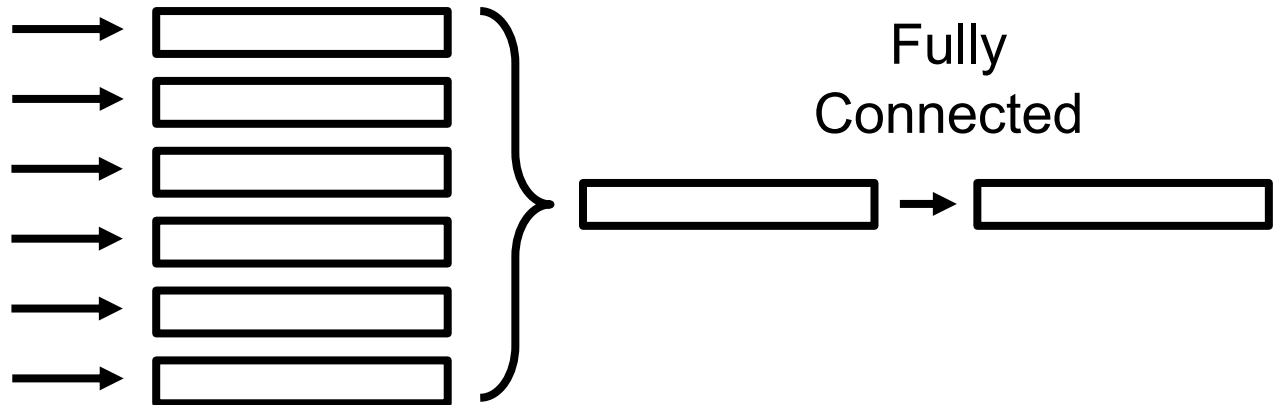
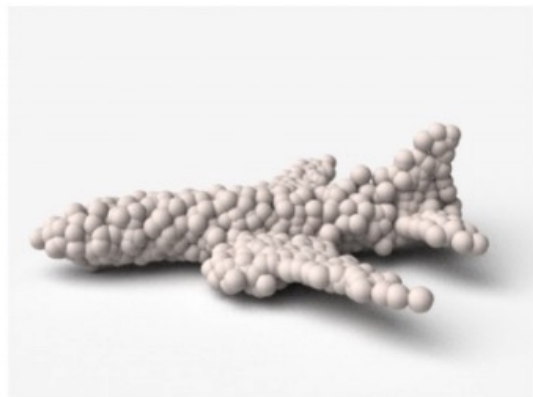
Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

Processing Pointcloud Inputs: PointNet

Run MLP on
each point

Max-Pool

Want to process
pointclouds as **sets**: order
should not matter



Input pointcloud:
 $P \times 3$

Point features:
 $P \times D$

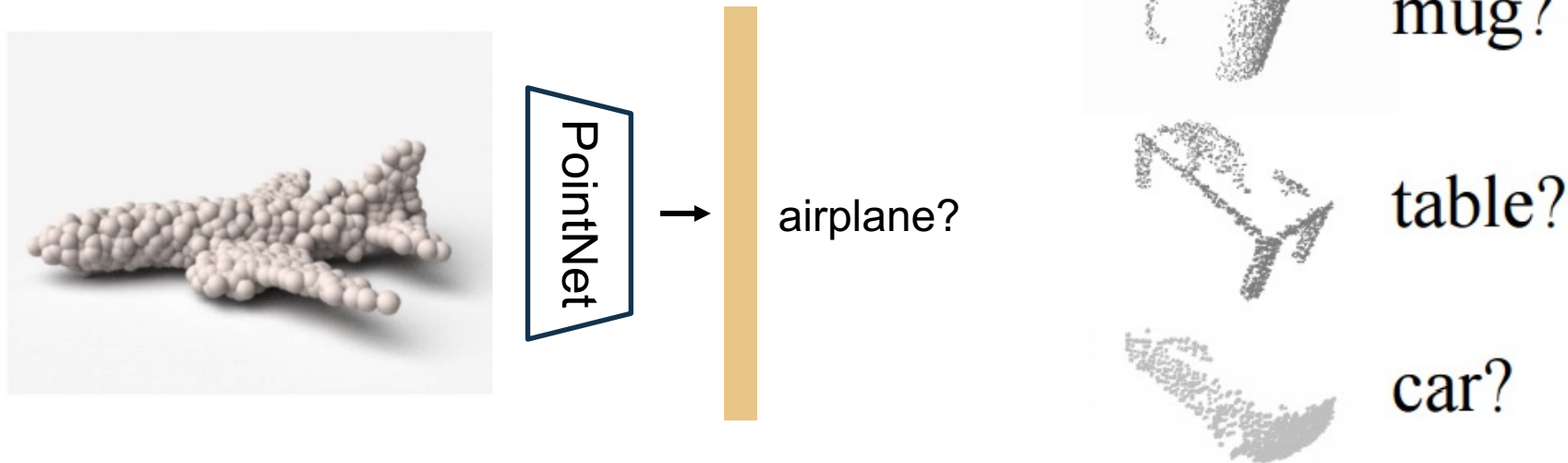
Pooled vector:
 D

Class score:
 C

Qi et al, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation", CVPR 2017

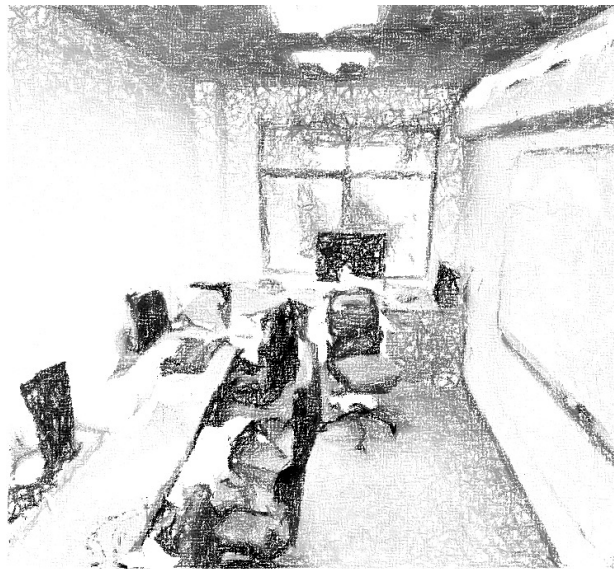
Qi et al, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space", NeurIPS 2017

PointNet applications:



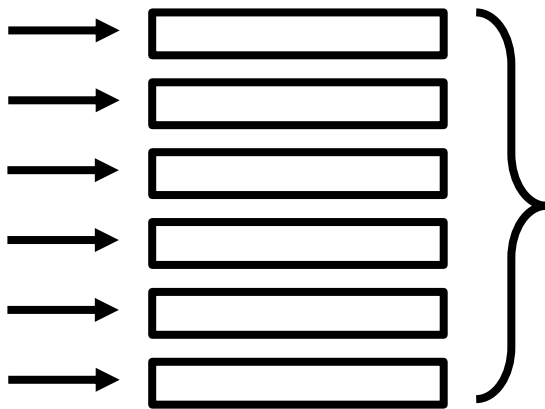
3D Object Classification

PointNet applications:



Point cloud inputs
 $P \times 3$

Run MLP on
each point

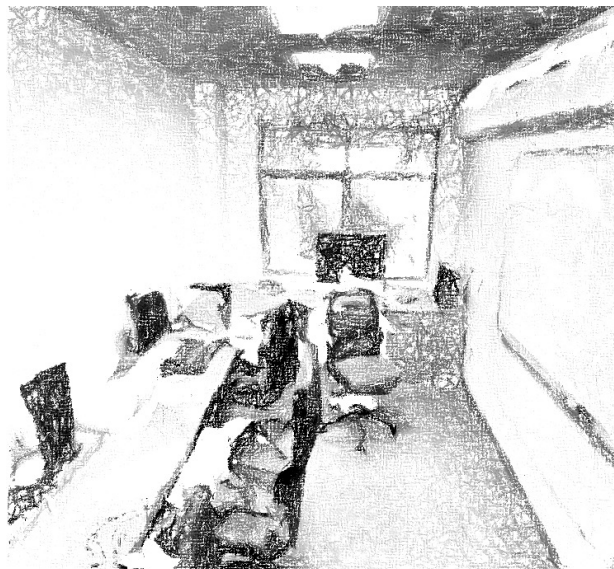


Point features:
 $P \times D$

Max-Pool



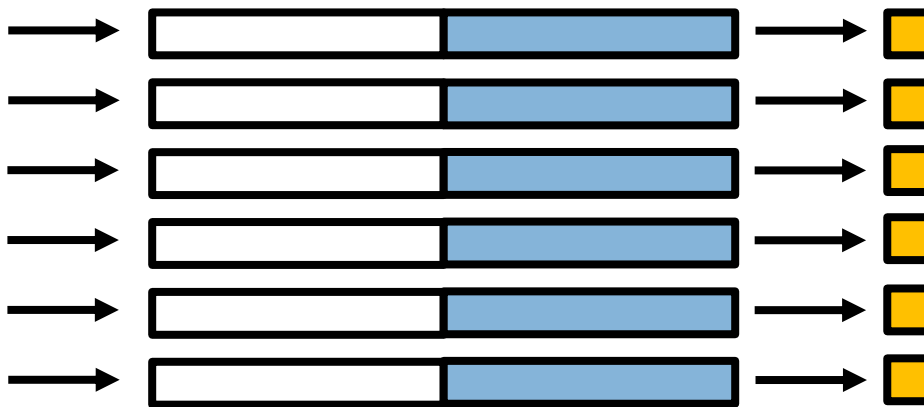
PointNet applications:



Point cloud inputs
 $P \times 3$

Run MLP on
each point

Copy&Concat



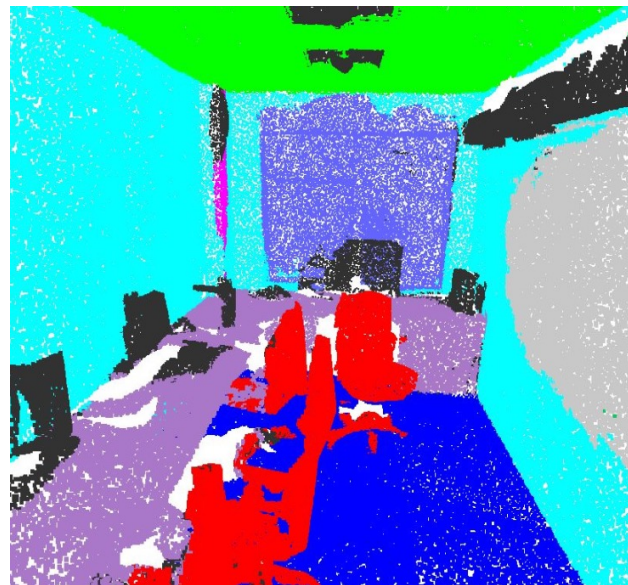
Point features:
 $P \times D$

Per-point
classification

PointNet applications:

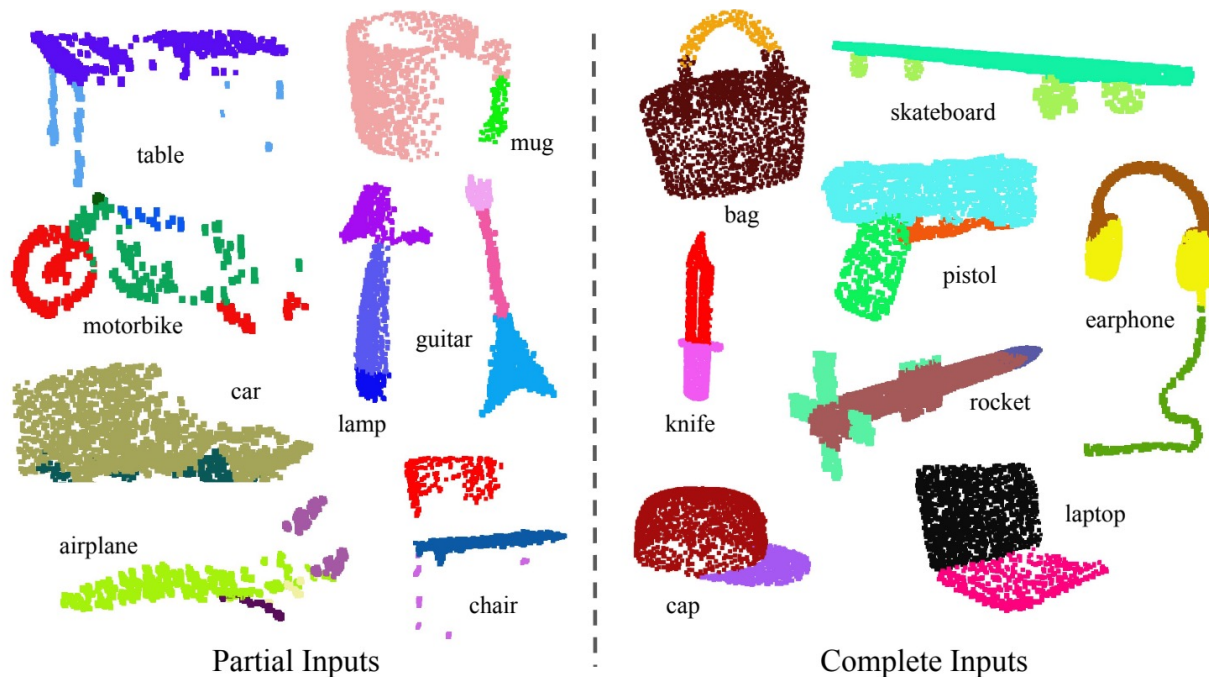


PointNet



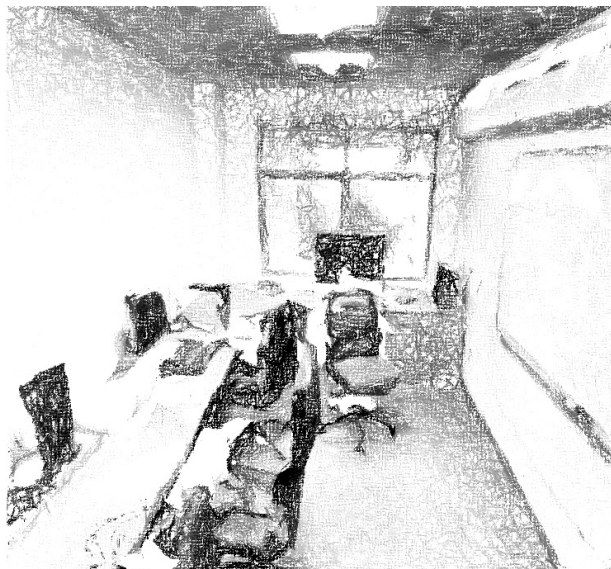
Semantic Segmentation on Point Clouds

PointNet applications:



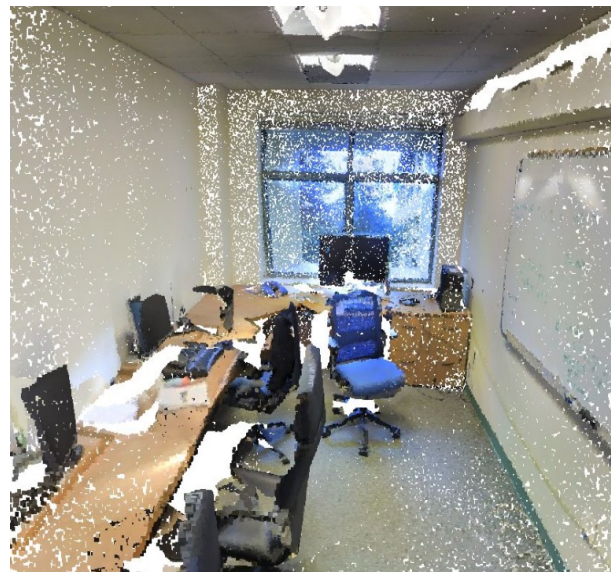
Object Part Segmentation

Sensor Fusion: Pointclouds + RGB



Point Clouds

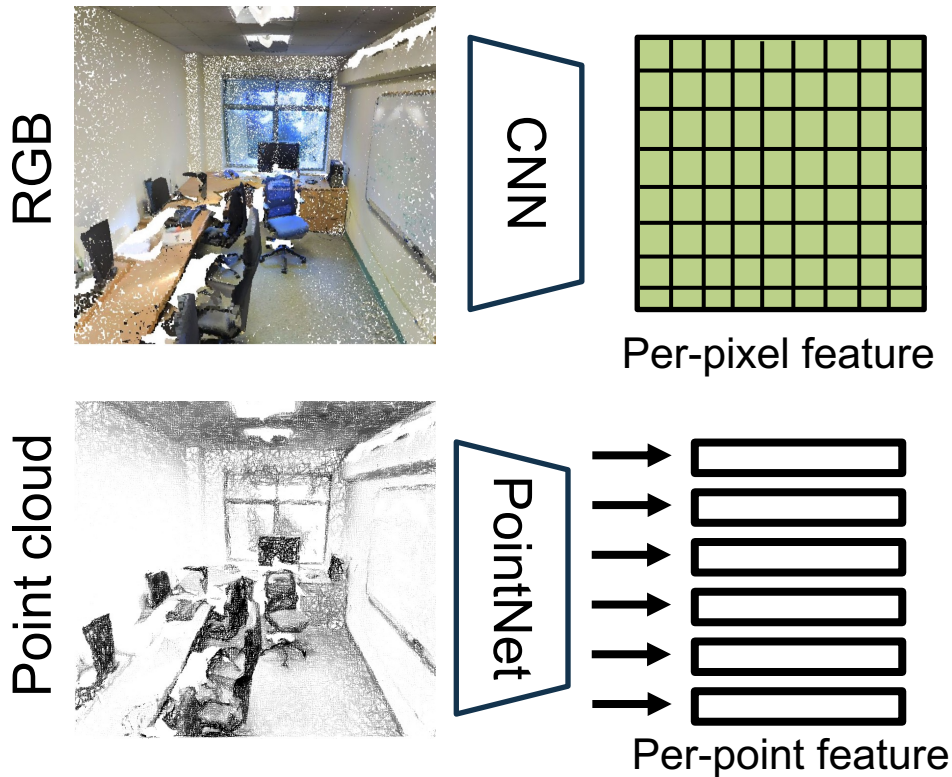
(x, y, z)



Point Clouds + RGB

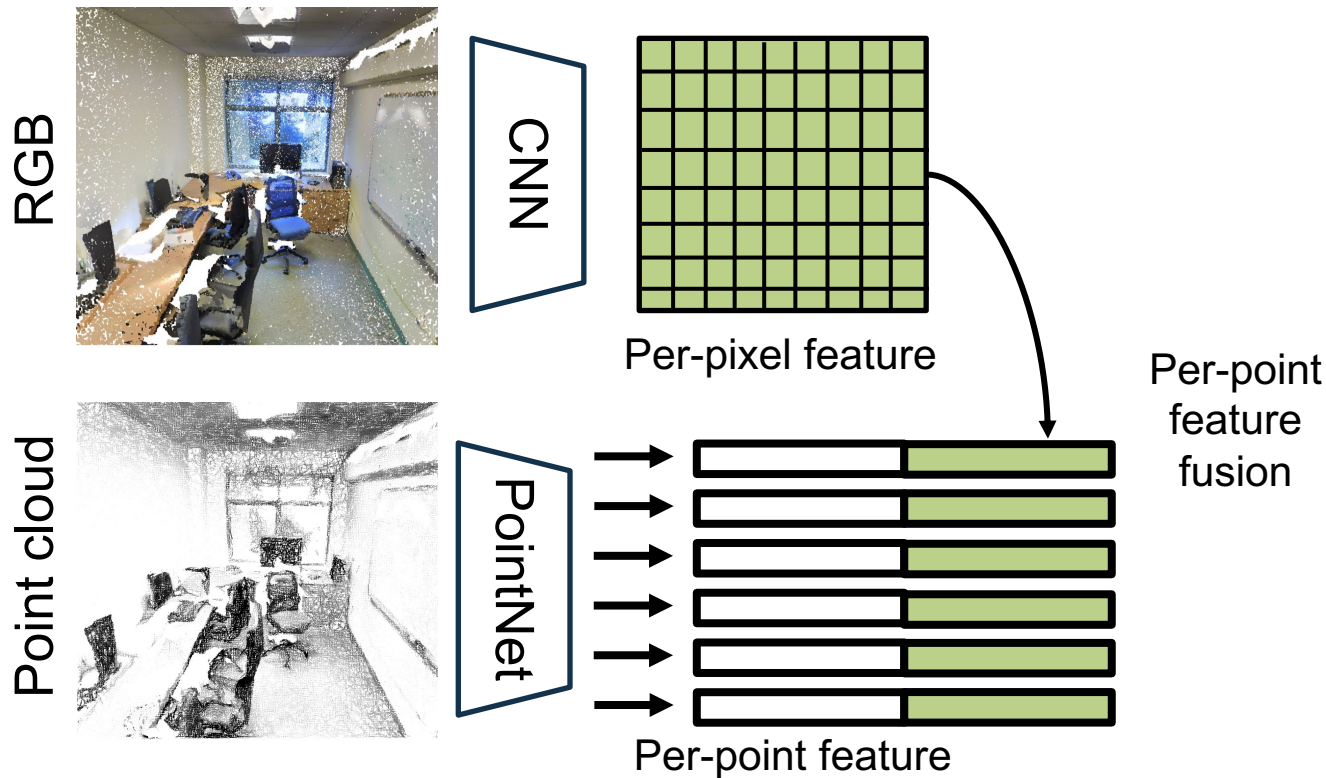
(x, y, z, r, g, b)

Processing Pointcloud+RGB: DenseFusion



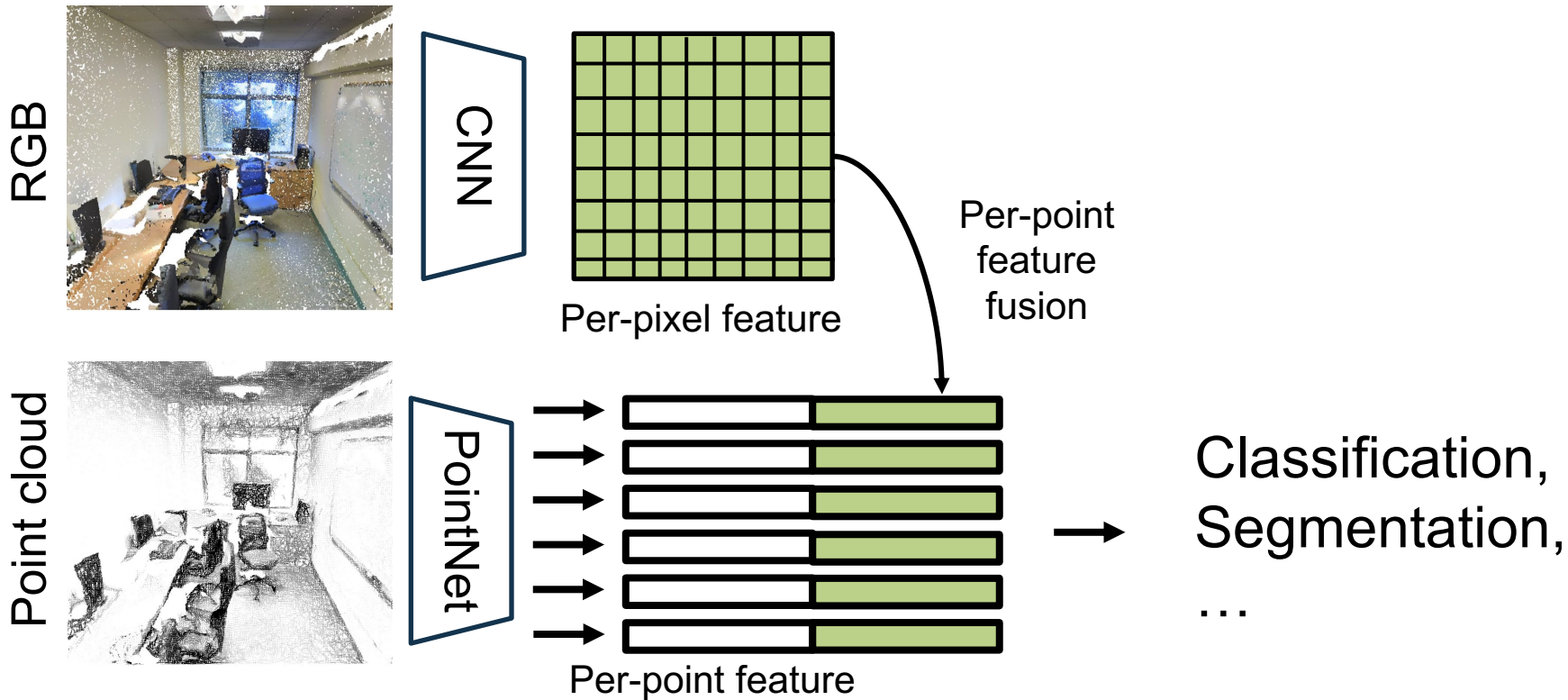
Wang et al, "DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion", CVPR 2019

Processing Pointcloud+RGB: DenseFusion



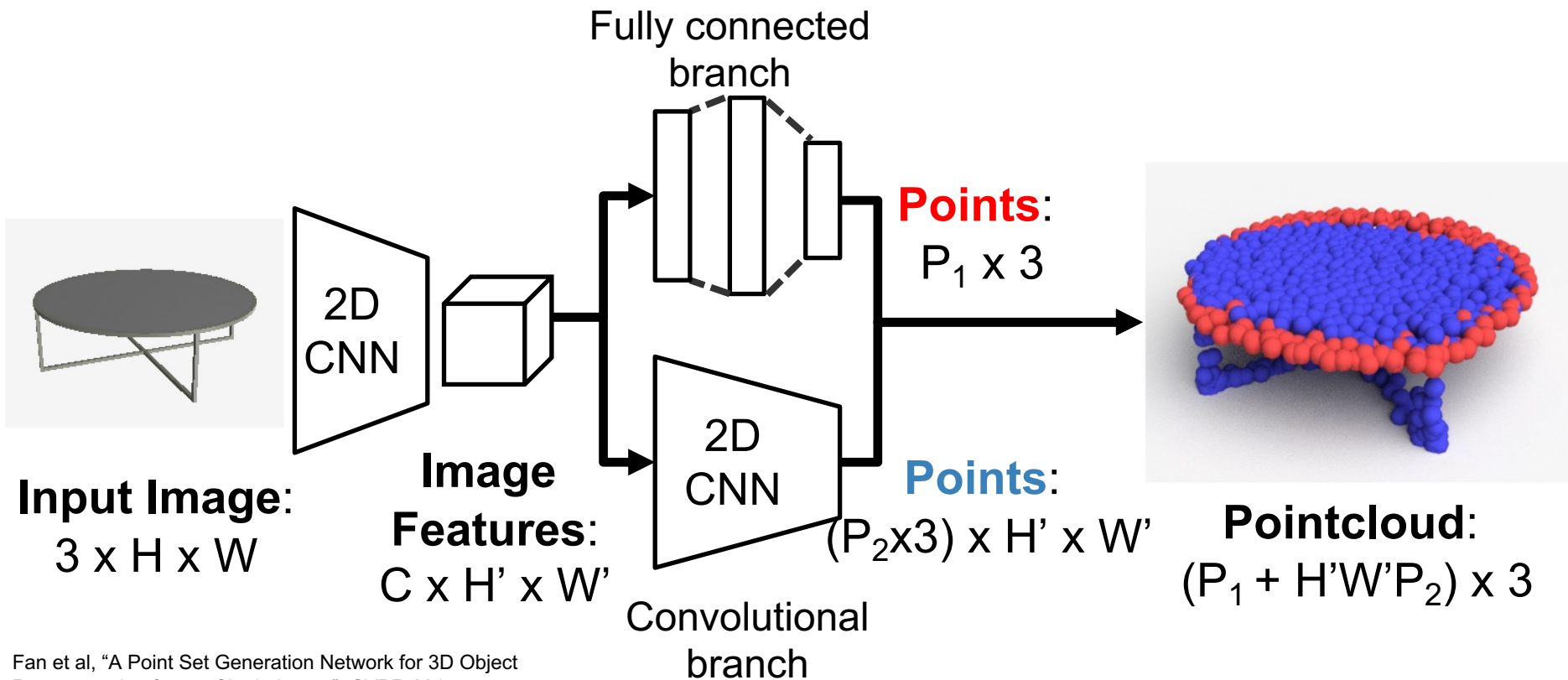
Wang et al, "DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion", CVPR 2019

Processing Pointcloud+RGB: DenseFusion



Wang et al, "DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion", CVPR 2019

Generating Pointcloud Outputs



Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

Predicting Point Clouds: Loss Function

We need a (differentiable) way to compare pointclouds **as sets**!

Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

Predicting Point Clouds: Loss Function

We need a (differentiable) way to compare pointclouds **as sets**!

Chamfer distance is the sum of L2 distance to each point's nearest neighbor in the other set

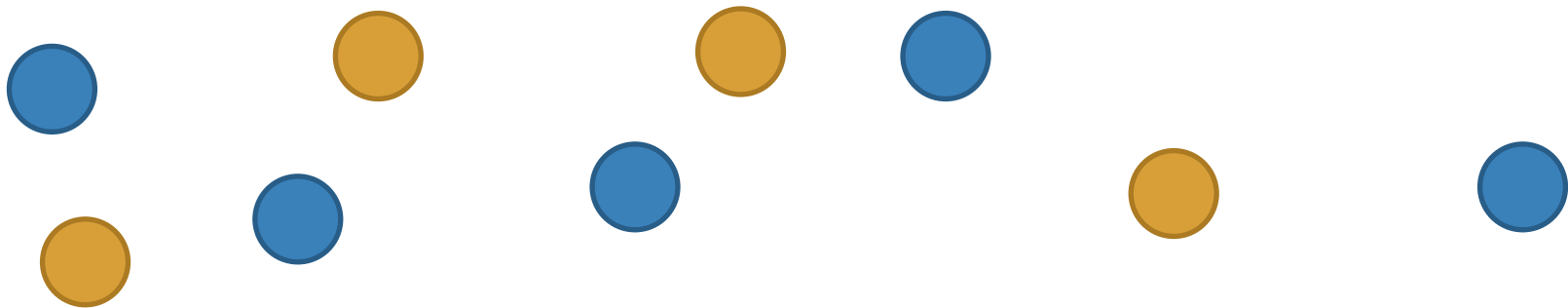
$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2^2$$

Predicting Point Clouds: Loss Function

We need a (differentiable) way to compare pointclouds **as sets**!

Chamfer distance is the sum of L2 distance to each point's nearest neighbor in the other set

$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2^2$$



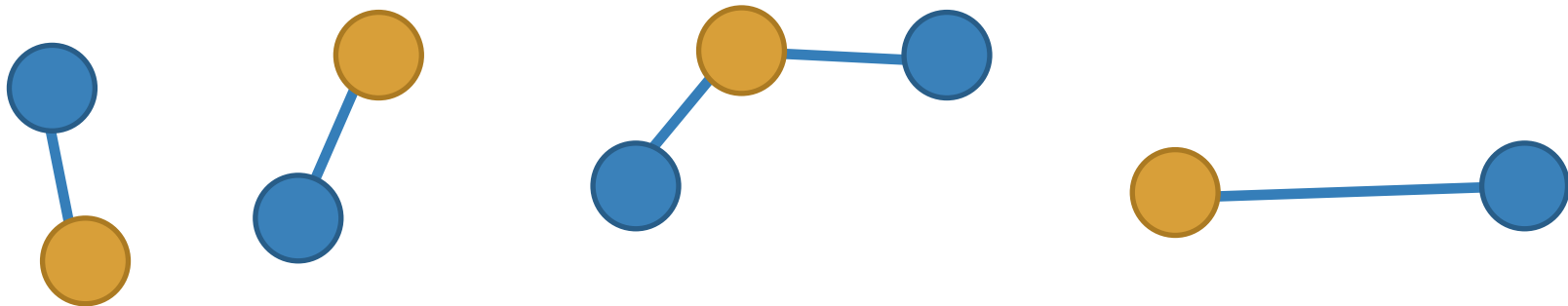
Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

Predicting Point Clouds: Loss Function

We need a (differentiable) way to compare pointclouds **as sets**!

Chamfer distance is the sum of L2 distance to each point's nearest neighbor in the other set

$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2^2$$



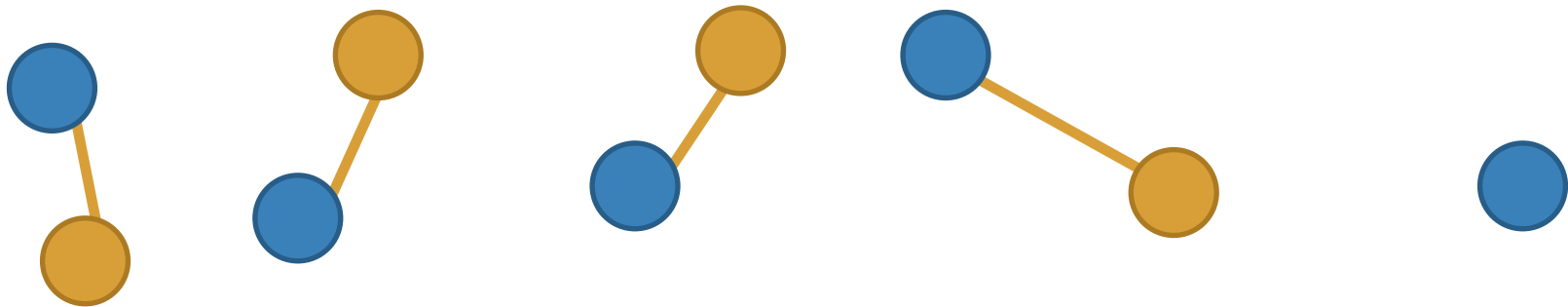
Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

Predicting Point Clouds: Loss Function

We need a (differentiable) way to compare pointclouds **as sets**!

Chamfer distance is the sum of L2 distance to each point's nearest neighbor in the other set

$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2^2$$

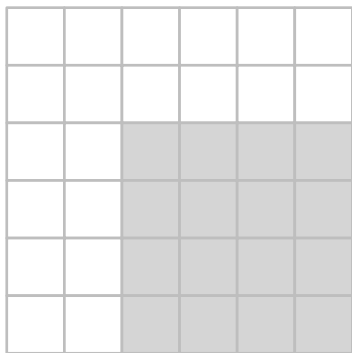


Fan et al, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image", CVPR 2017

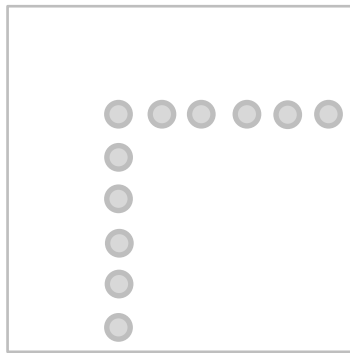
3D Shape Representations



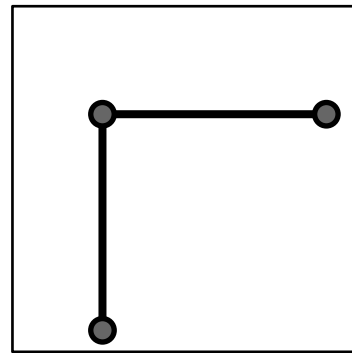
Depth
Map



Voxel
Grid



Pointcloud



Mesh



Implicit
Surface

3D Shape Representations: Triangle Mesh

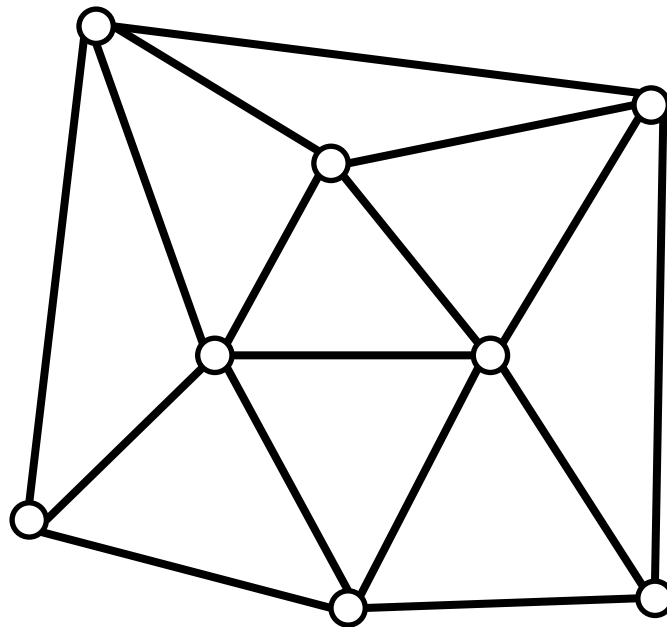
Represent a 3D shape as a set of triangles

Vertices: Set of V points in 3D space

Faces: Set of triangles over the vertices

(+) Standard representation for graphics

(+) Explicitly represents 3D shapes



3D Shape Representations: Triangle Mesh

Represent a 3D shape as a set of triangles

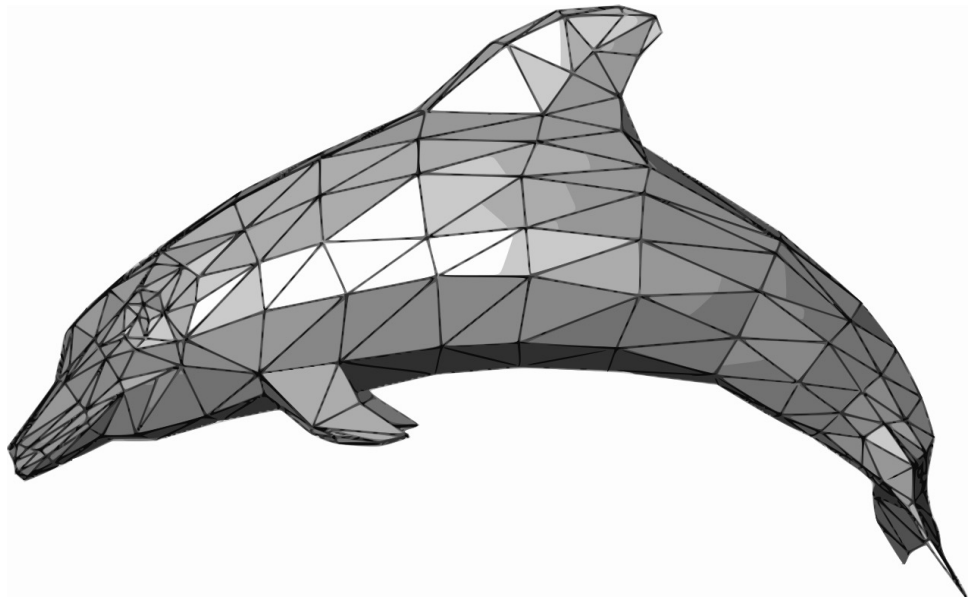
Vertices: Set of V points in 3D space

Faces: Set of triangles over the vertices

(+) Standard representation for graphics

(+) Explicitly represents 3D shapes

(+) Adaptive: Can represent flat surfaces very efficiently, can allocate more faces to areas with fine detail



[Dolphin image](#) is in the public domain

3D Shape Representations: Triangle Mesh

Represent a 3D shape as a set of triangles

Vertices: Set of V points in 3D space

Faces: Set of triangles over the vertices

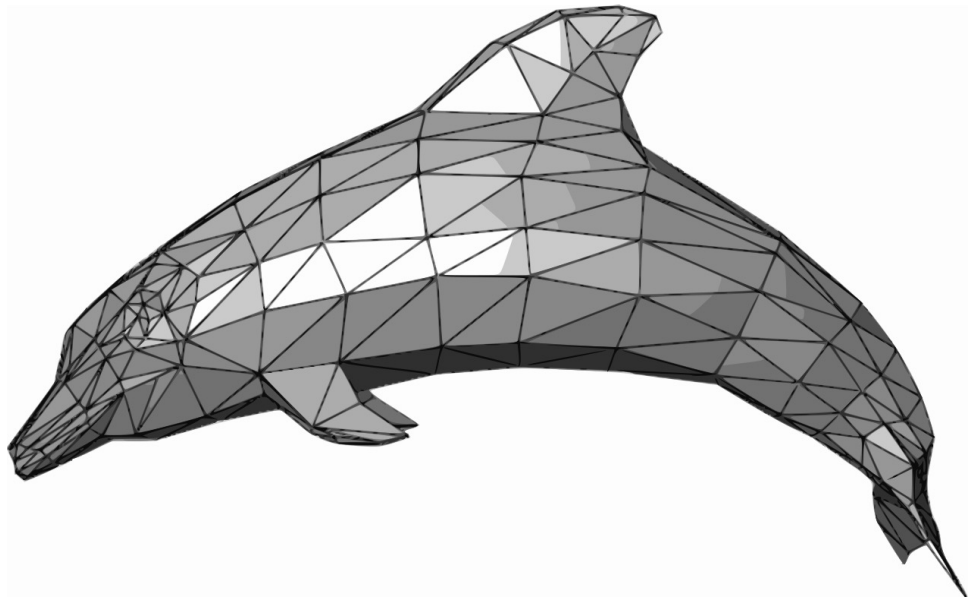
(+) Standard representation for graphics

(+) Explicitly represents 3D shapes

(+) Adaptive: Can represent flat surfaces very efficiently, can allocate more faces to areas with fine detail

(+) Can attach data on verts and interpolate over the whole surface: RGB colors, texture coordinates, normal vectors, etc.

(-) Nontrivial to process with neural networks

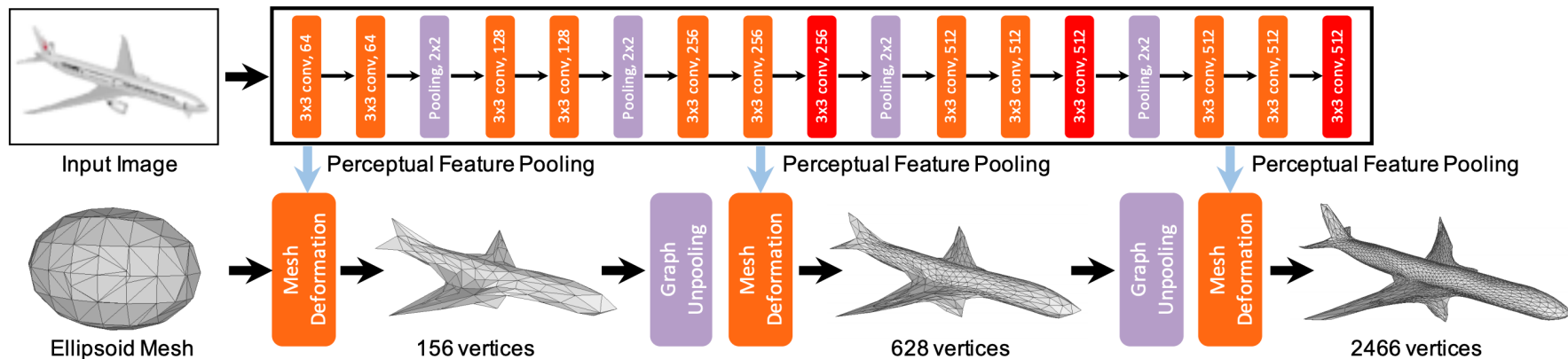


UV mapping figure is
licensed under [CC-BY-SA](#)
3.0. Figure slightly
reorganized.

Predicting Meshes: Pixel2Mesh

Input: Single RGB Image of an object

Output: Triangle mesh for the object



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

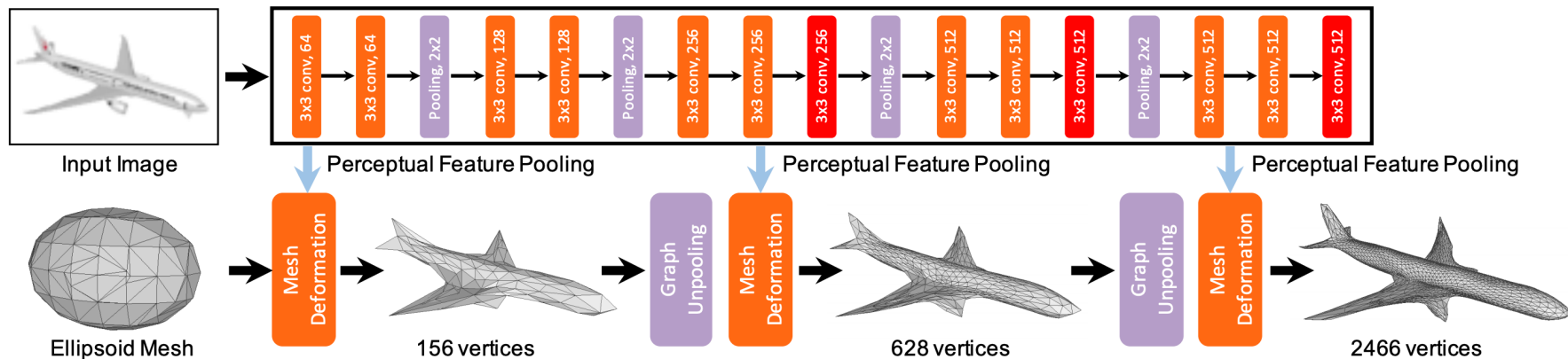
Predicting Meshes: Pixel2Mesh

Input: Single RGB
Image of an object

Key ideas:

Iterative Refinement
Graph Convolution
Vertex Aligned-Features
Chamfer Loss Function

Output: Triangle
mesh for the object



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

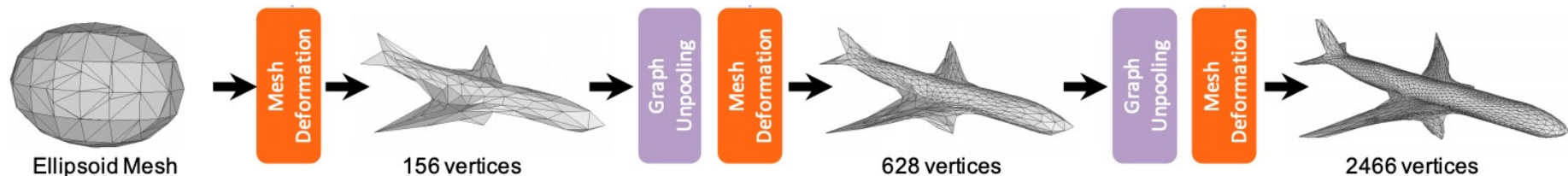
Predicting Triangle Meshes: Iterative Refinement

Idea #1: Iterative mesh refinement

Start from initial ellipsoid mesh

Network predicts offsets for each vertex

Repeat.



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

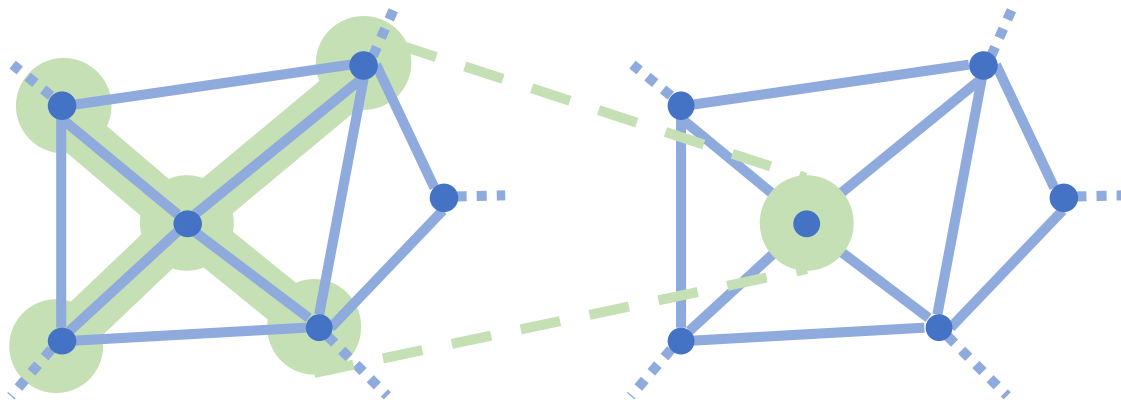
Predicting Triangle Meshes: Graph Convolution

$$f'_i = W_0 f_i + \sum_{j \in N(i)} W_1 f_j$$

Vertex v_i has feature f_i

New feature f'_i for vertex v_i depends on feature of neighboring vertices $N(i)$

Use same weights W_0 and W_1 to compute all outputs

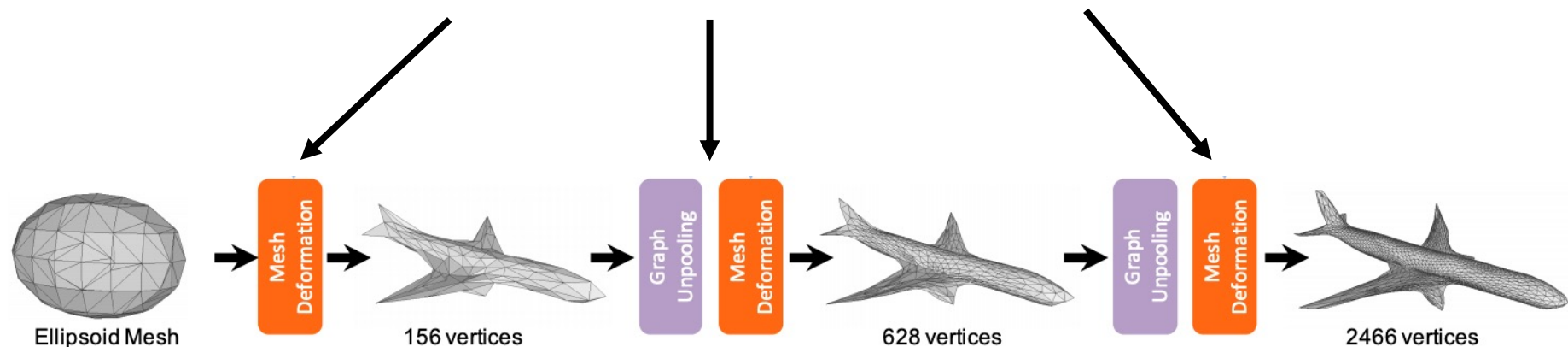


Input: Graph with a feature vector at each vertex

Output: New feature vector for each vertex

Predicting Triangle Meshes: Graph Convolution

Each of these blocks consists of a stack of **graph convolution layers** operating on edges of the mesh

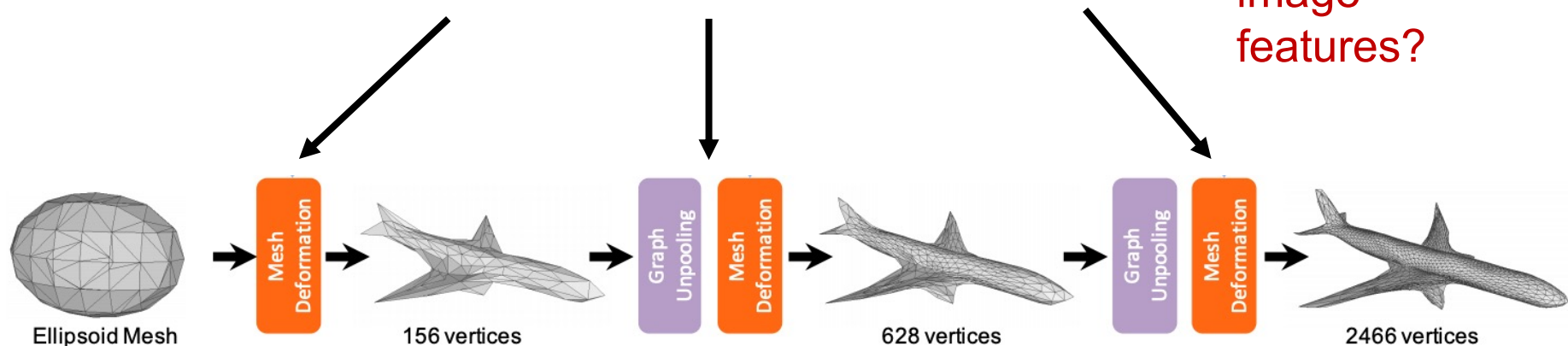


Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Predicting Triangle Meshes: Graph Convolution

Each of these blocks consists of a stack of **graph convolution layers** operating on edges of the mesh

Problem: How to incorporate image features?



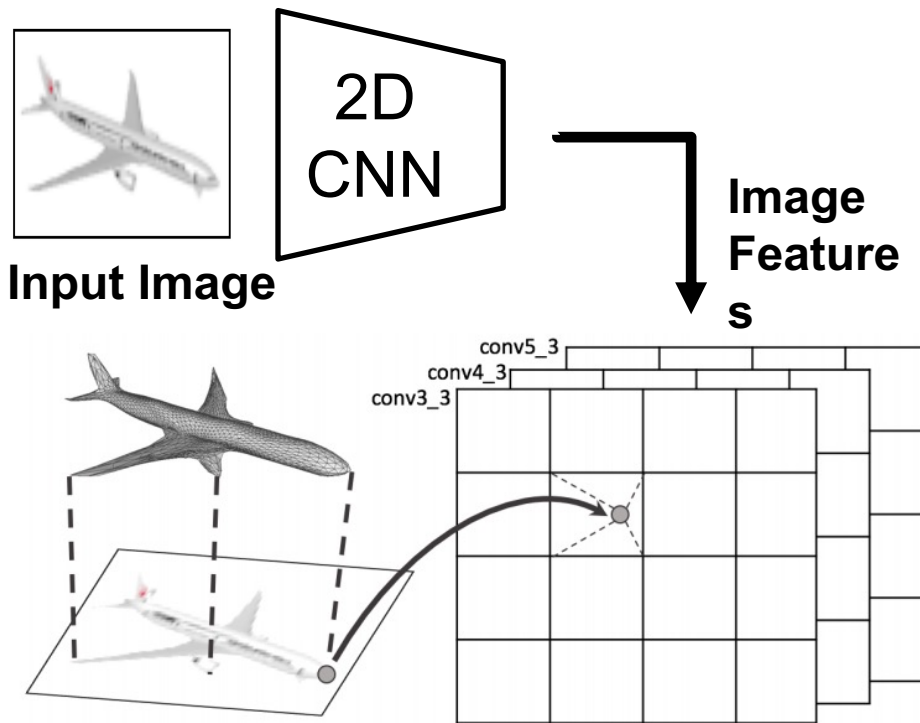
Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Predicting Triangle Meshes: Vertex-Aligned Features

Idea #2: Aligned vertex features

For each vertex of the mesh:

- Use camera information to project onto image plane
- Use bilinear interpolation to sample a CNN feature



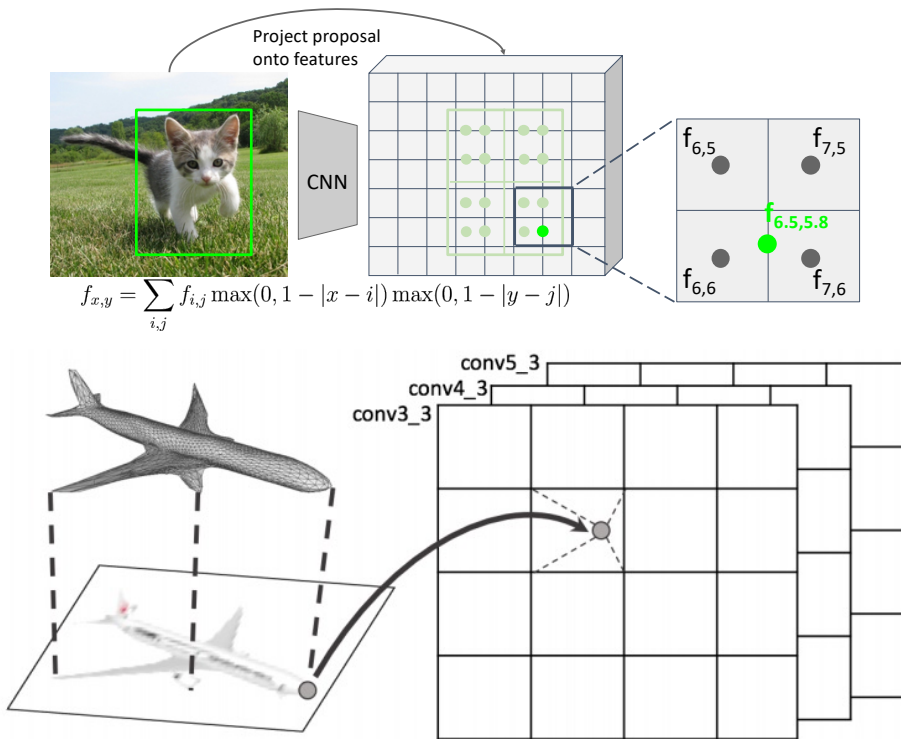
Predicting Triangle Meshes: Vertex-Aligned Features

Idea #2: Aligned vertex features

For each vertex of the mesh:

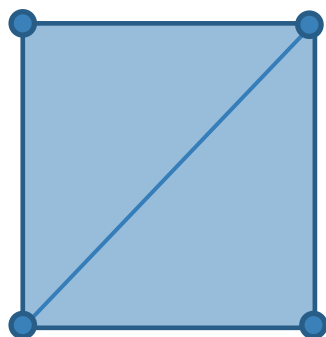
- Use camera information to project onto image plane
- Use bilinear interpolation to sample a CNN feature

Similar to RoI-Align operation from detection: maintains alignment between input image and feature vectors



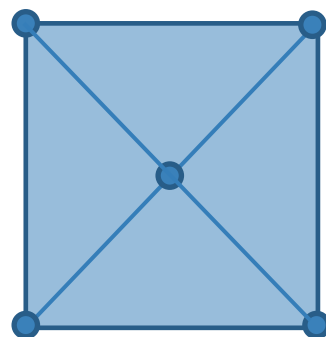
Predicting Meshes: Loss Function

The same shape can be represented with different meshes – how can we define a loss between predicted and ground-truth mesh?



Prediction

VS



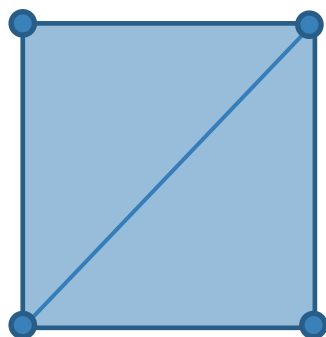
Ground-Truth

Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Predicting Meshes: Loss Function

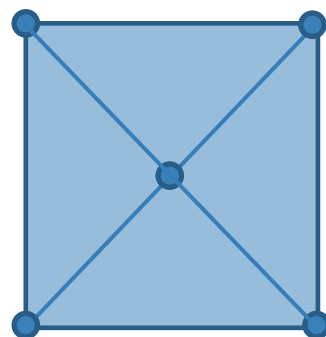
The same shape can be represented with different meshes – how can we define a loss between predicted and ground-truth mesh?

Idea: Convert meshes to pointclouds, then compute loss



Prediction

VS



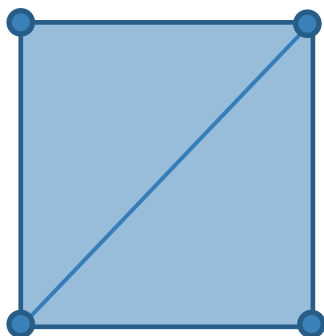
Ground-Truth

Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Predicting Meshes: Loss Function

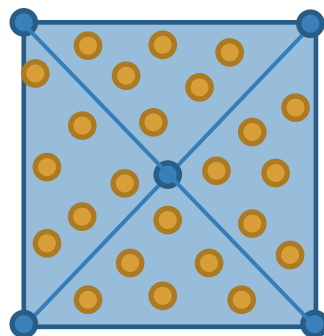
The same shape can be represented with different meshes – how can we define a loss between predicted and ground-truth mesh?

Idea: Convert meshes to pointclouds, then compute loss



Prediction

vs



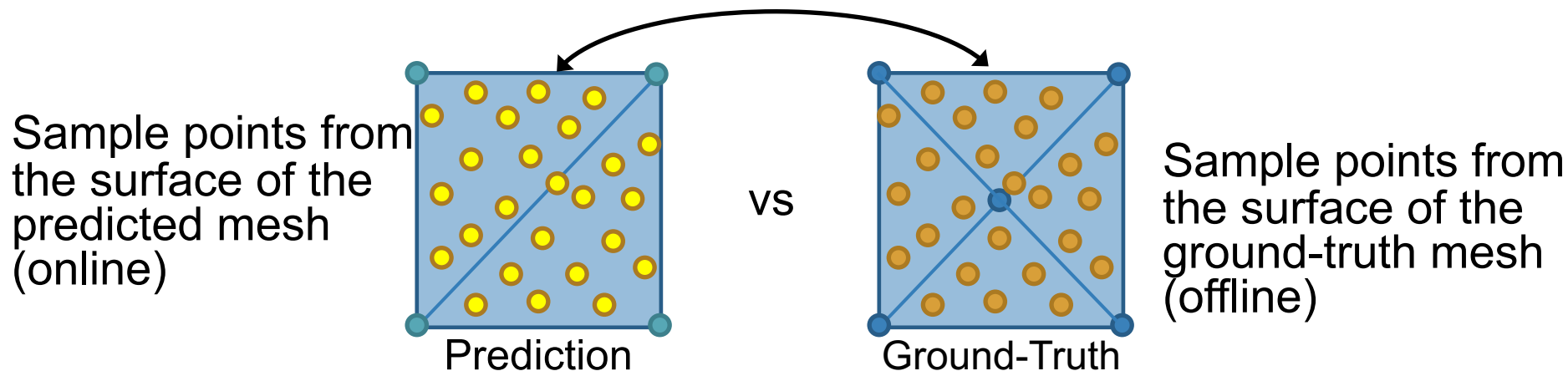
Ground-Truth

Sample points from the surface of the ground-truth mesh (offline)

Predicting Meshes: Loss Function

The same shape can be represented with different meshes – how can we define a loss between predicted and ground-truth mesh?

Loss = Chamfer distance between **predicted verts** and **ground-truth samples**



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

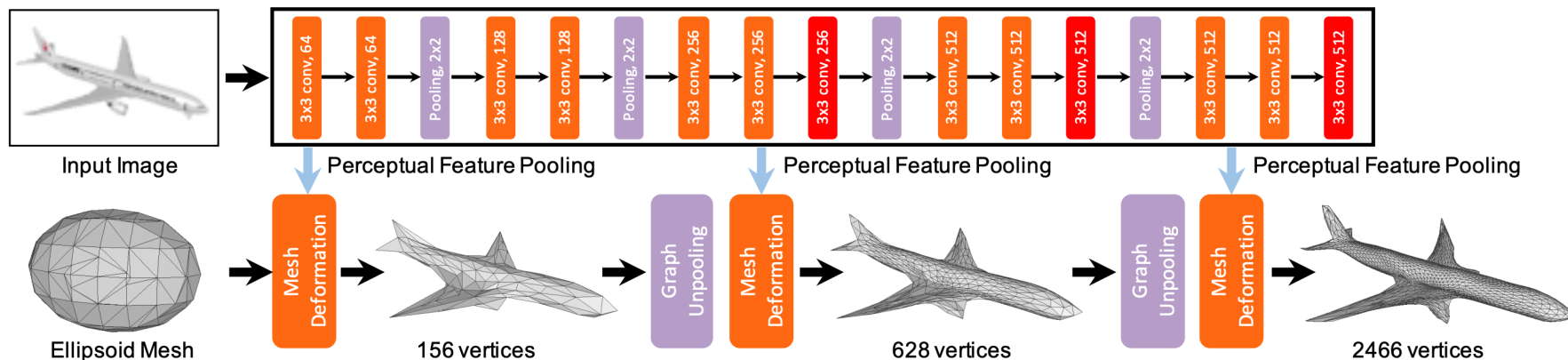
Predicting Meshes: Pixel2Mesh

Input: Single RGB
Image of an object

Key ideas:

Iterative Refinement
Graph Convolution
Vertex Aligned-Features
Chamfer Loss Function

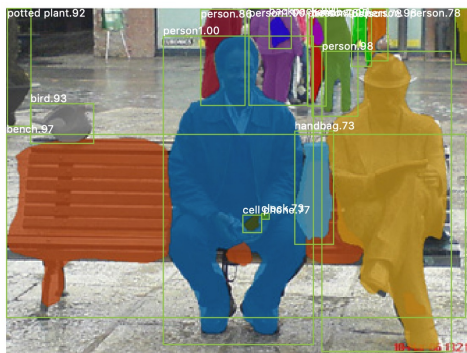
Output: Triangle
mesh for the object



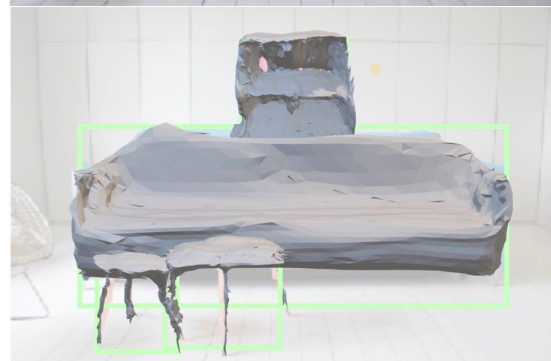
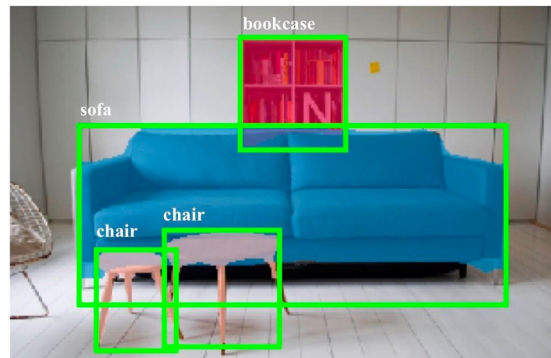
Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

3D Shape Prediction: Mesh R-CNN

Mask R-CNN:
2D Image -> 2D shapes



Mesh R-CNN:
2D Image -> **Triangle Meshes**



He, Gkioxari, Dollár,
and Girshick, "Mask
R-CNN", ICCV 2017

Gkioxari, Malik, and
Johnson, "Mesh R-CNN",
ICCV 2019

Mesh R-CNN: Task

Input: Single RGB image

Output:

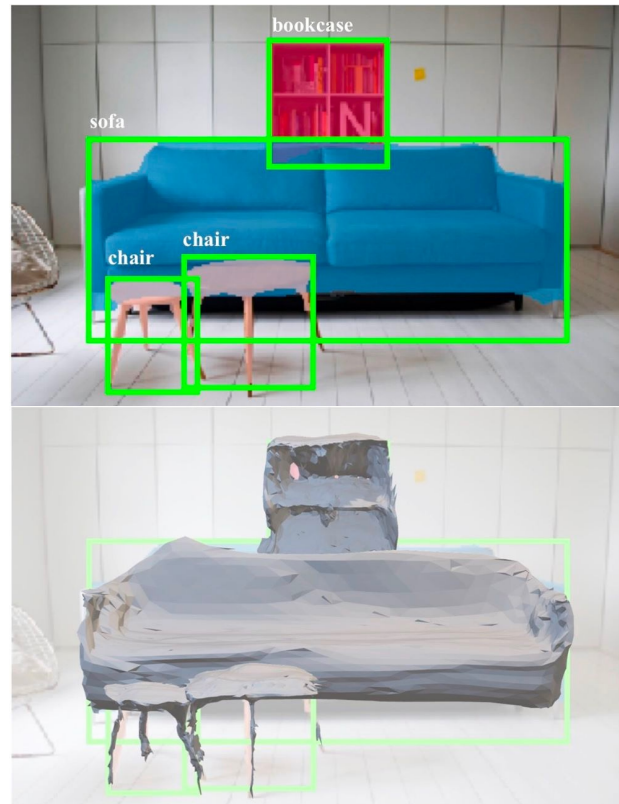
A set of detected objects

For each object:

- Bounding box
- Category label
- Instance segmentation
- 3D triangle mesh

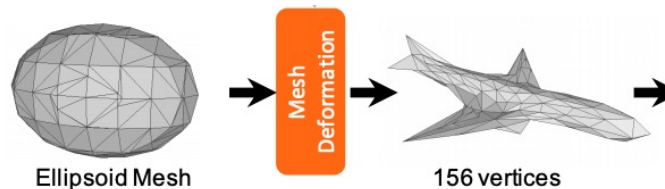
Mask R-
CNN

Mesh head



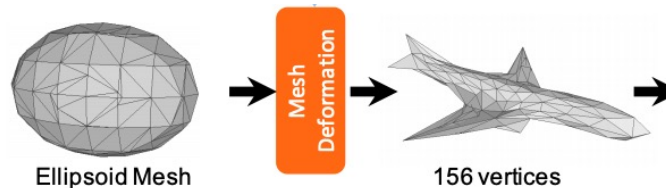
Mesh R-CNN: Hybrid 3D shape representation

Mesh deformation gives good results, but the topology (verts, faces, genus, connected components) fixed by the initial mesh

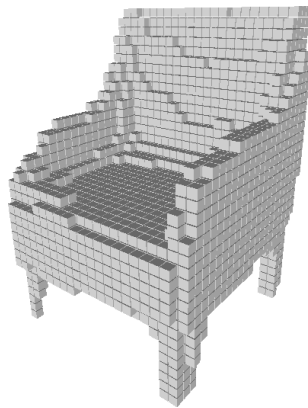


Mesh R-CNN: Hybrid 3D shape representation

Mesh deformation gives good results, but the topology (verts, faces, genus, connected components) fixed by the initial mesh



Mesh R-CNN: Use voxel predictions to create initial mesh prediction!



Mesh R-CNN Pipeline

Input image



Mesh R-CNN Pipeline

Input image



2D object recognition



Mesh R-CNN Pipeline

Input image



2D object recognition



3D object voxels

Mesh R-CNN Pipeline

Input image



2D object recognition



3D object meshes

3D object voxels

Mesh R-CNN: ShapeNet Results



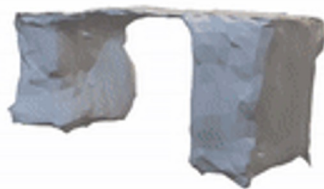
Datasets for 3D Objects

- Large-scale Synthetic Objects: ShapeNet, 3M models
- ModelNet: absorbed by ShapeNet
- ShapeNetCore: 51.3K models in 55 categories



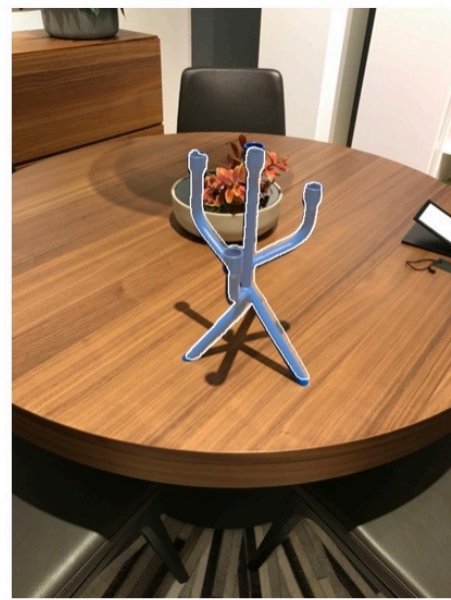
Chang et al. ShapeNet. arXiv 2015
Wu et al. 3D ShapeNets. CVPR 2015

Mesh R-CNN: Pix3D Results



Pix3D

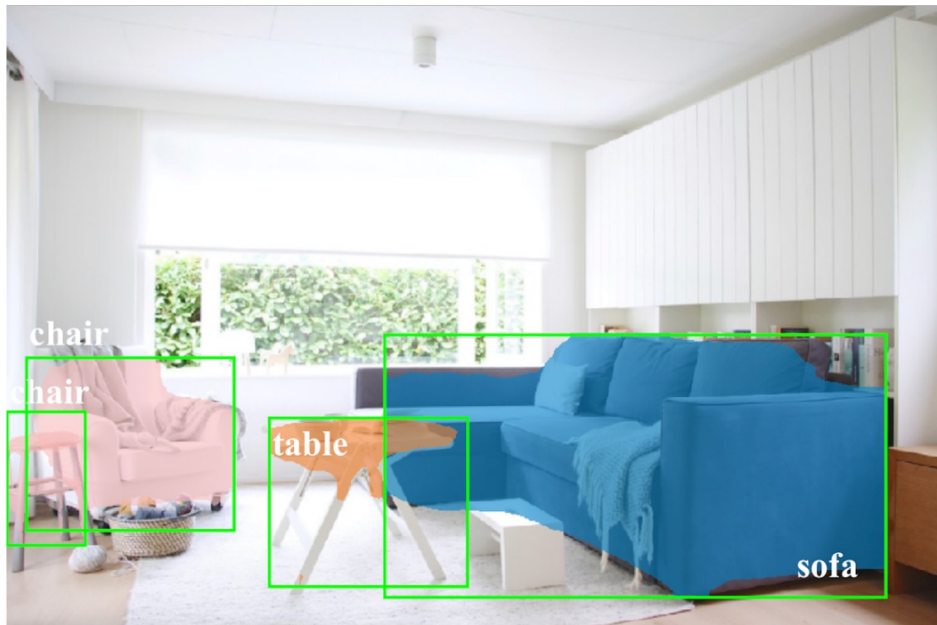
- 10,069 images
- 395 shapes (IKEA furniture + 3D scan)



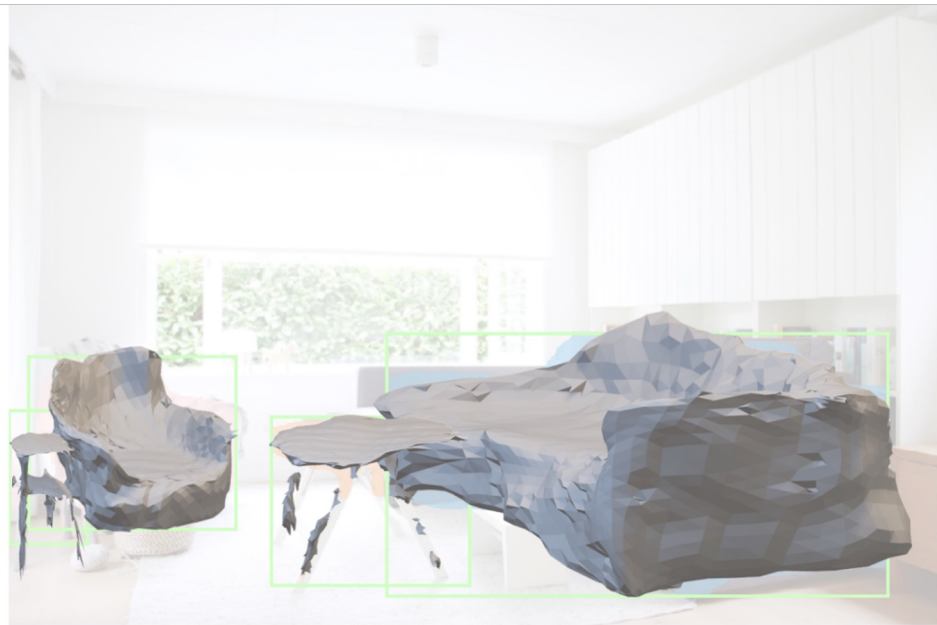
Sun et al. CVPR 2018, building upon Lim et al. ICCV 2013

Mesh R-CNN: Pix3D Results

Predicting many objects per scene



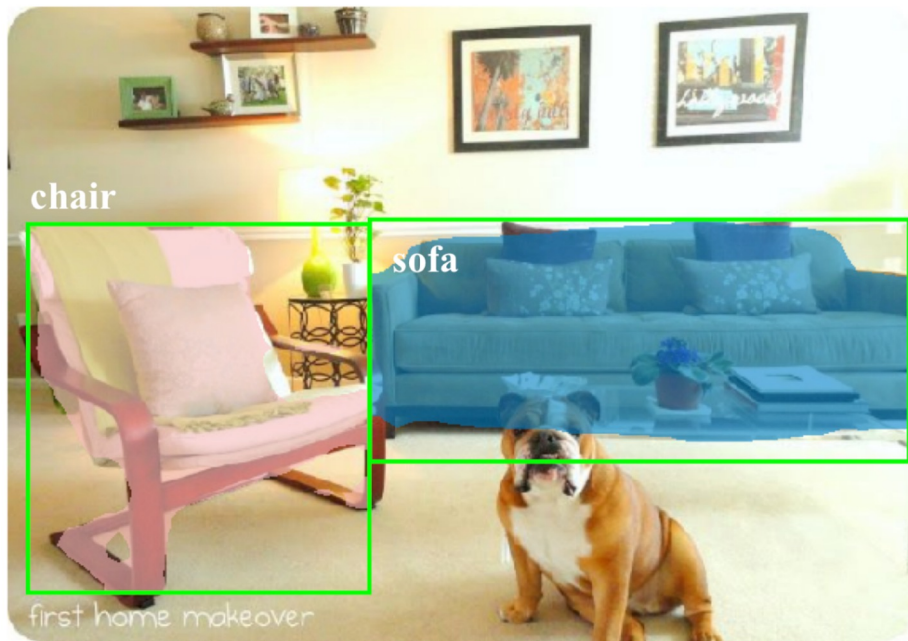
Box & Mask Predictions



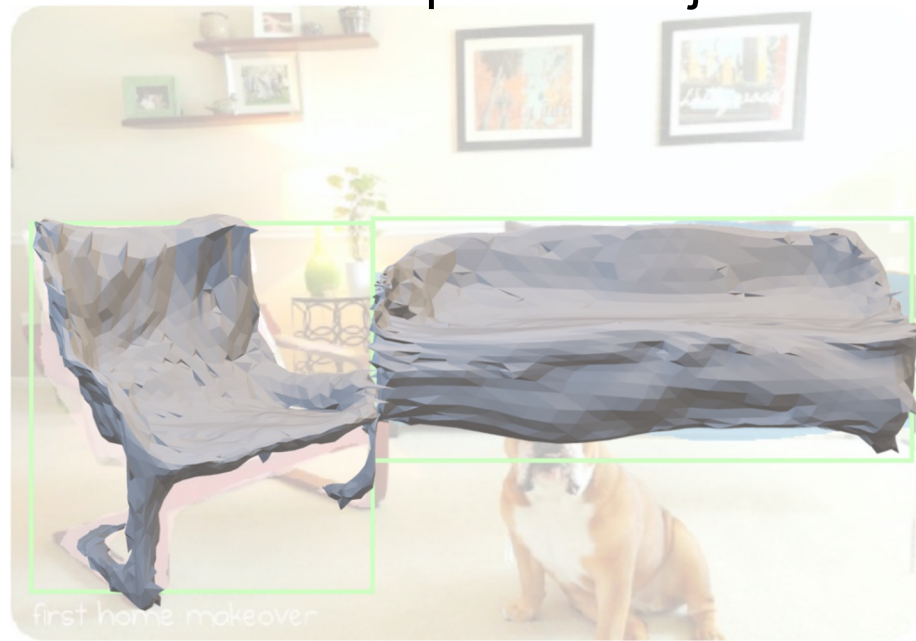
Mesh Predictions

Mesh R-CNN: Pix3D Results

Amodal completion: predict
occluded parts of objects



Box & Mask Predictions



Mesh Predictions

Mesh R-CNN: Pix3D Results

Segmentation failures
propagate to meshes



Box & Mask Predictions



Mesh Predictions

Link to WordNet Taxonomy Alignment+Symmetry

Part Hierarchy

Part Correspondences



WordNet synset

Swivel chair: a chair that swivels on its base

Hypernyms: chair > seat > furniture > ...

Part meronyms: backrest, seat, base

Sister terms: armchair, barber chair, ...

ImageNet



Swivel chair



Backrest



Dim: 50 x 45 x 5 cm
Material: foam, fabric
Mass: 5 Kg
Function: support

Seat



Base



Leg



Wheel

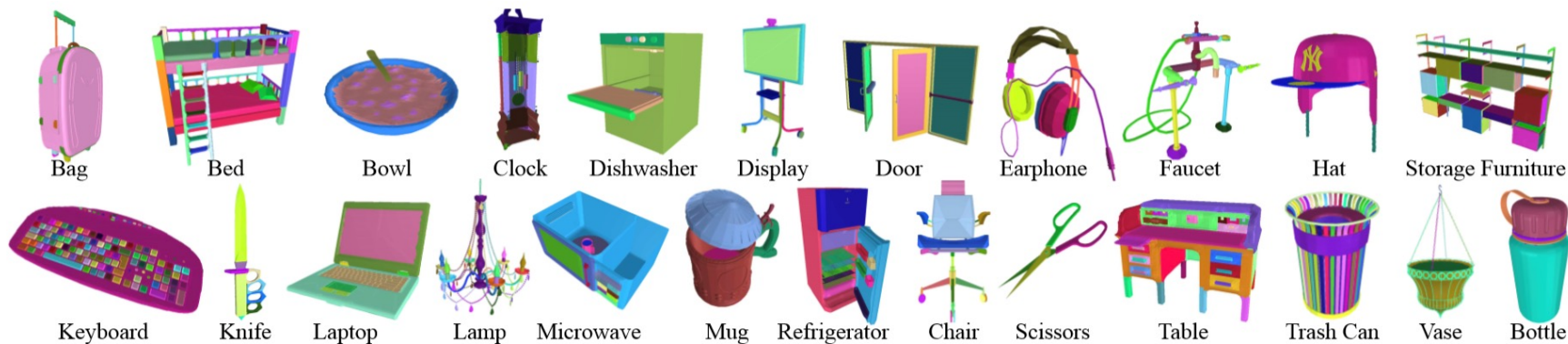


Figure from the ShapeNet paper, Chang et al. arXiv 2015

Datasets for 3D Object Parts

Fine-grained Parts: PartNet

- Fine-grained (+mobility)
- Instance-level
- Hierarchical



Mo et al. CVPR 2019

Slide credit: Hao Su

Physical Interaction with Articulated Objects

**300+ door
annotations**

**support
articulated
objects**

(cabinets, doors, fridge,
oven, window etc.)



<http://svl.stanford.edu/igibson/>

BEHAVIOR-1K Project

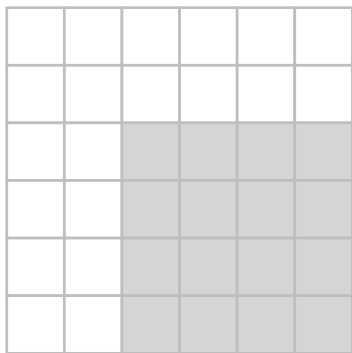


<https://behavior.stanford.edu/behavior-1k>

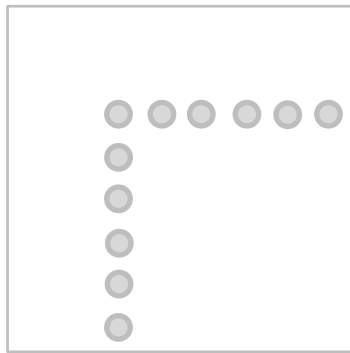
3D Shape Representations



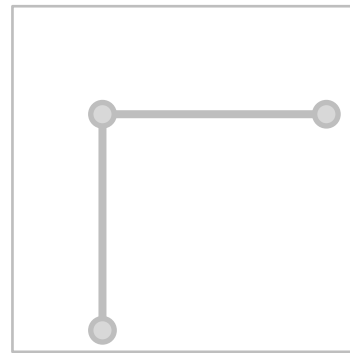
Depth
Map



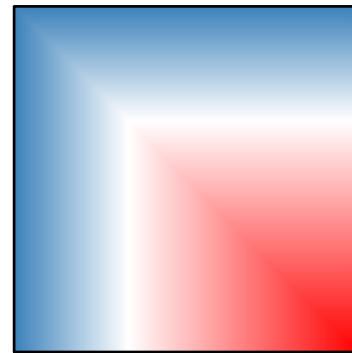
Voxel
Grid



Pointcloud



Mesh



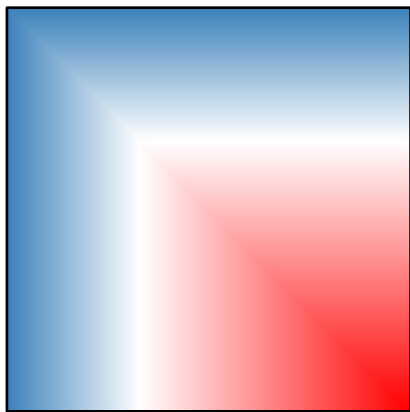
Implicit
Surface

3D Shape Representations: Implicit Functions

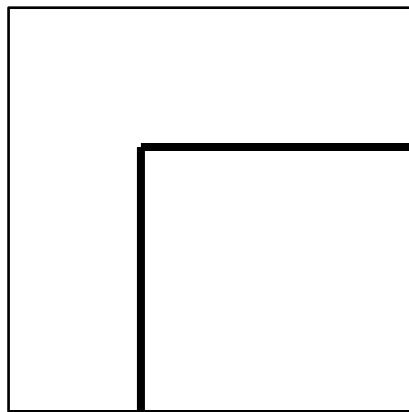
Learn a function to classify arbitrary 3D points as inside / outside the shape

$$o : \mathbb{R}^3 \rightarrow \{0, 1\}$$

The surface of the 3D object is the level set $\{x : o(x) = 1/2\}$



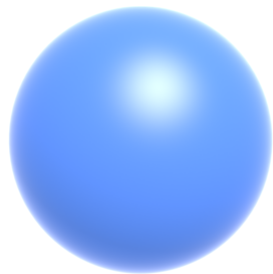
Implicit function



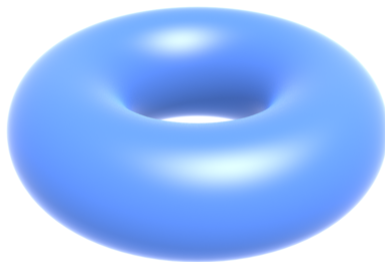
Explicit Shape

Algebraic Surfaces (Implicit)

Surface is zero set of a polynomial in x, y, z



$$x^2 + y^2 + z^2 = 1$$



$$(R - \sqrt{x^2 + y^2})^2 + z^2 = r^2$$

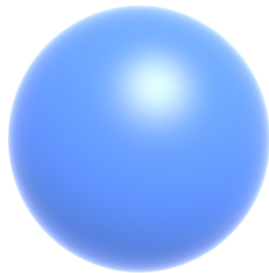


$$(x^2 + \frac{9y^2}{4} + z^2 - 1)^3 = x^2 z^3 + \frac{9y^2 z^3}{80}$$

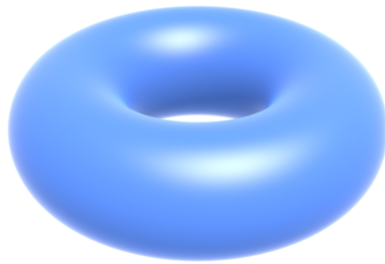
Slide credit: Ren Ng

Algebraic Surfaces (Implicit)

Surface is zero set of a polynomial in x, y, z



$$x^2 + y^2 + z^2 = 1$$



$$(R - \sqrt{x^2 + y^2})^2 + z^2 = r^2$$



$$(x^2 + \frac{9y^2}{4} + z^2 - 1)^3 = x^2 z^3 + \frac{9y^2 z^3}{80}$$

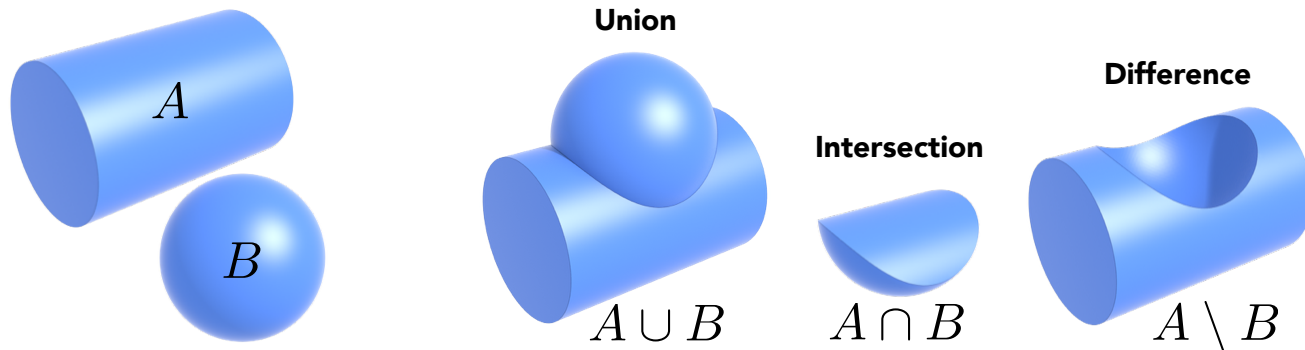


More complex shapes?

Slide credit: Ren Ng

Constructive Solid Geometry (Implicit)

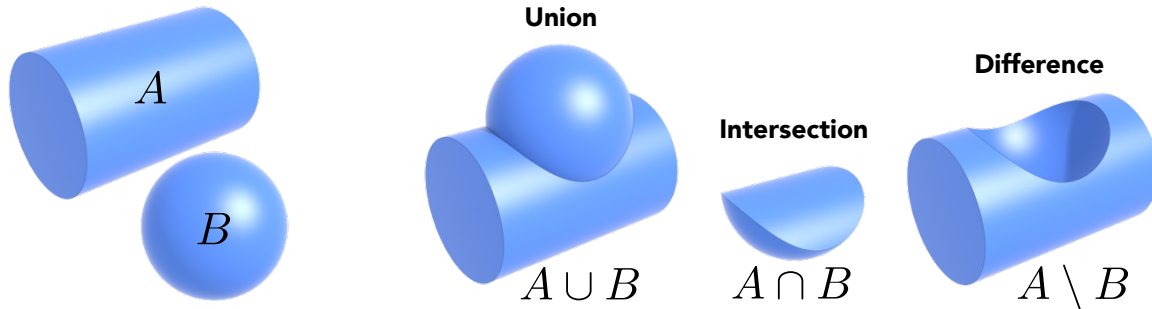
Combine implicit geometry via Boolean operations



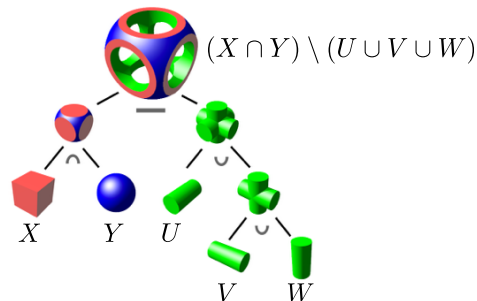
Slide credit: Ren Ng

Constructive Solid Geometry (Implicit)

Combine implicit geometry via Boolean operations



Boolean expressions:



CS184/284A

Ren Ng

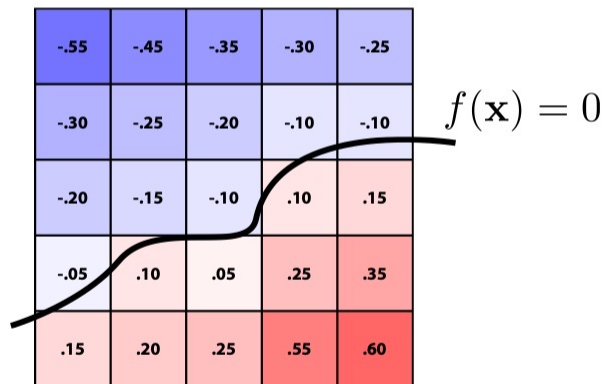
Slide credit: Ren Ng

Level Set Methods (Implicit)

Implicit surfaces have some nice features (e.g., merging/splitting)

But, hard to describe complex shapes in closed form

Alternative: store a grid of values approximating function



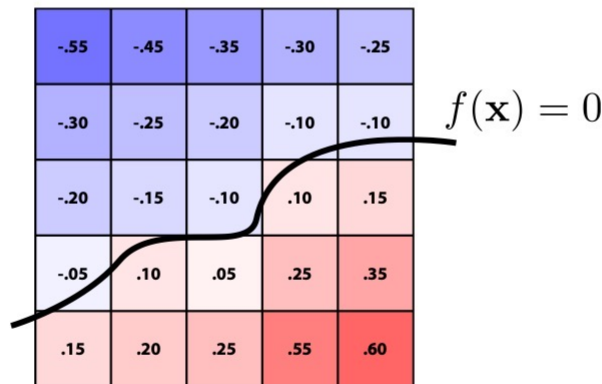
Slide credit: Ren Ng

Level Set Methods (Implicit)

Implicit surfaces have some nice features (e.g., merging/splitting)

But, hard to describe complex shapes in closed form

Alternative: store a grid of values approximating function

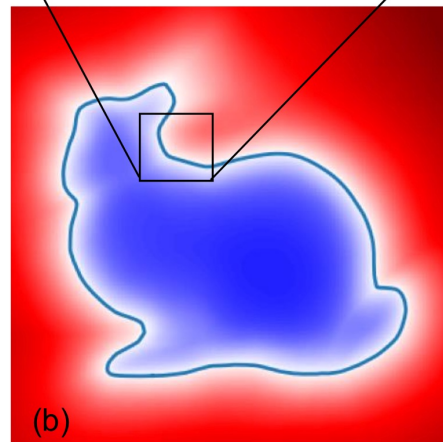
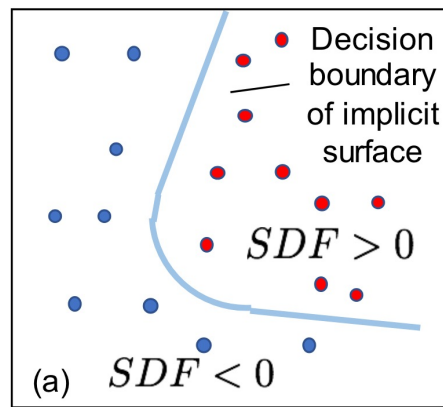


Surface is found where interpolated values equal zero

Provides much more explicit control over shape (like a texture)

Slide credit: Ren Ng

DeepSDF



Park et al., "DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation", CVPR 2019

Fei-Fei Li, Ehsan Adeli, Chen Wang

Lecture 18 - 87

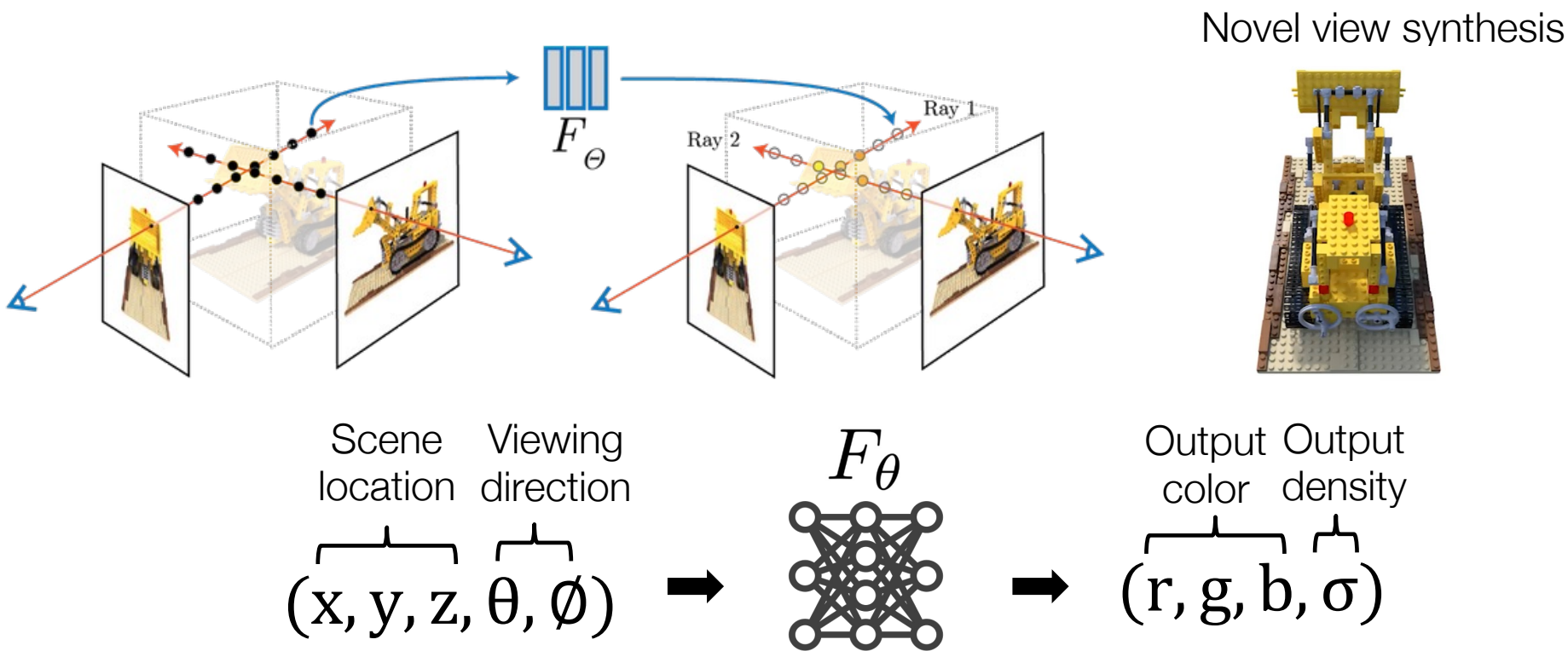
Jun 4, 2024

Neural Radiance Fields (NeRF)



Mildenhall et al, "Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020

NeRF: Representing Scenes as Neural Radiance Fields



Mildenhall et al, "Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020

Neural Radiance Fields



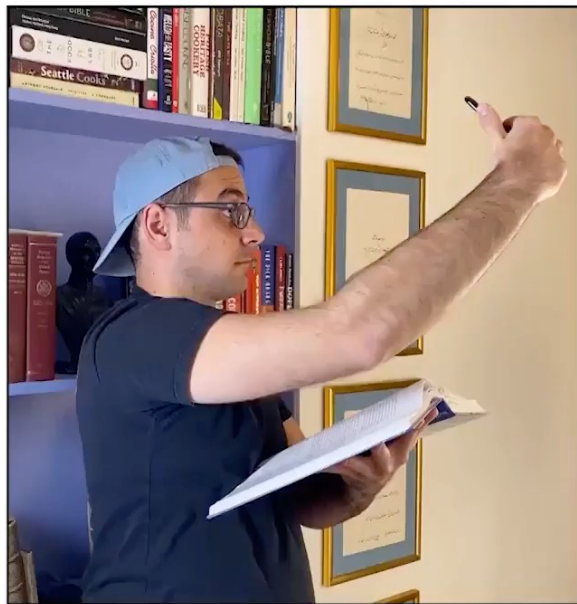
Mildenhall et al, "Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020

Neural Radiance Fields



Mildenhall et al, "Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020

Dynamic NeRF: Deformable Scenes



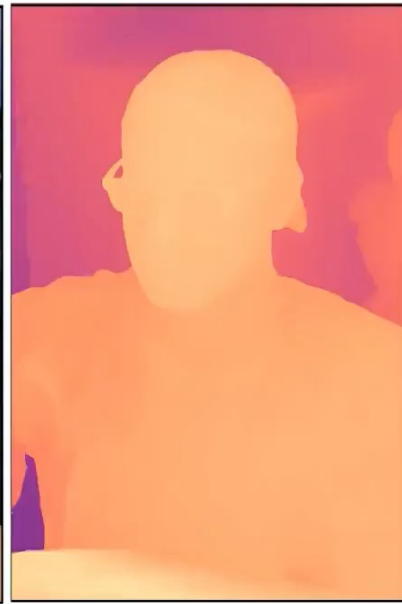
(a) Capture Process



(b) Input



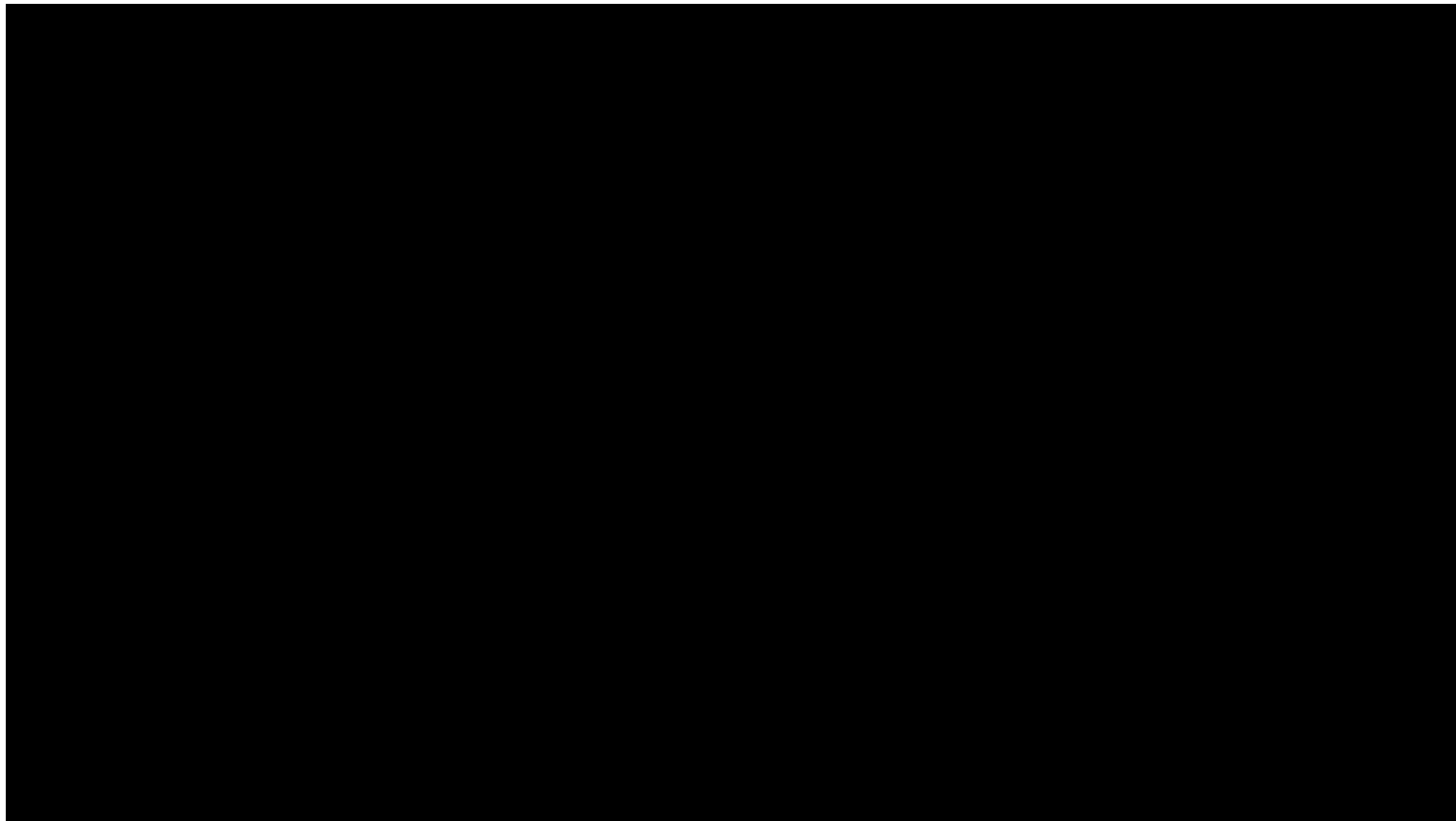
(c) Nerfie



(d) Nerfie Depth

Park et al, "Nerfies: Deformable Neural Radiance Fields", ICCV 2021

RawNeRF: High-Dynamic Range Imagery



Mildenhall et al, "NeRF in the Dark: High Dynamic Range View Synthesis from Noisy Raw Images", CVPR 2022

BlockNeRF: A Neighborhood of San Francisco



Tancik et al, "Block-NeRF: Scalable Large Scene Neural View Synthesis", CVPR 2022

Main Problem: Very slow!

Training: 1-2 days on a V100 GPU, for just a single scene!

Inference: Sampling an image from a trained model:
(256 x 256 pixels) x (224 samples per pixel)
= 14.6M forward passes through MLP

3D Gaussian Splatting

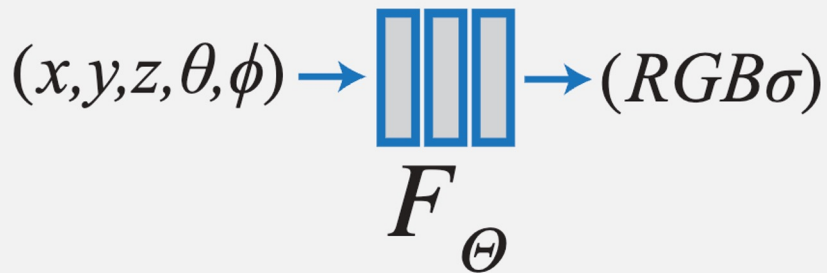


Kerbl and
Kopanas et al.,
2023

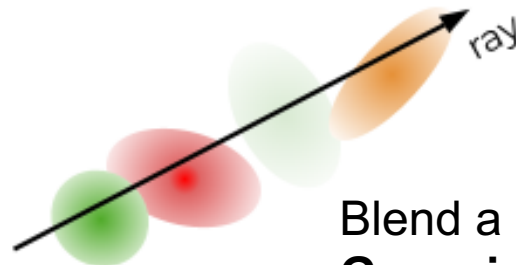
3D Gaussian Splatting



Query a **continuous MLP** along the ray



NeRF



Blend a **discrete set of Gaussians** along the ray



3D Gaussian Splatting

3D Gaussian Splatting



- Fitting is pretty **slow** (**several hours** with best GPUs)
- Rendering is **pretty slow** (~10 **seconds per frame** for moderate resolution)

F_{Θ}

NeRF



- **Fast** scene fitting (a few minutes)
- **Real-time** rendering
- High-quality visuals

3D Gaussians

3D Gaussian Splatting

Dynamic 3D Gaussians



Luiten et al, "Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis", 3DV 2024

3D Gaussian Splatting SLAM



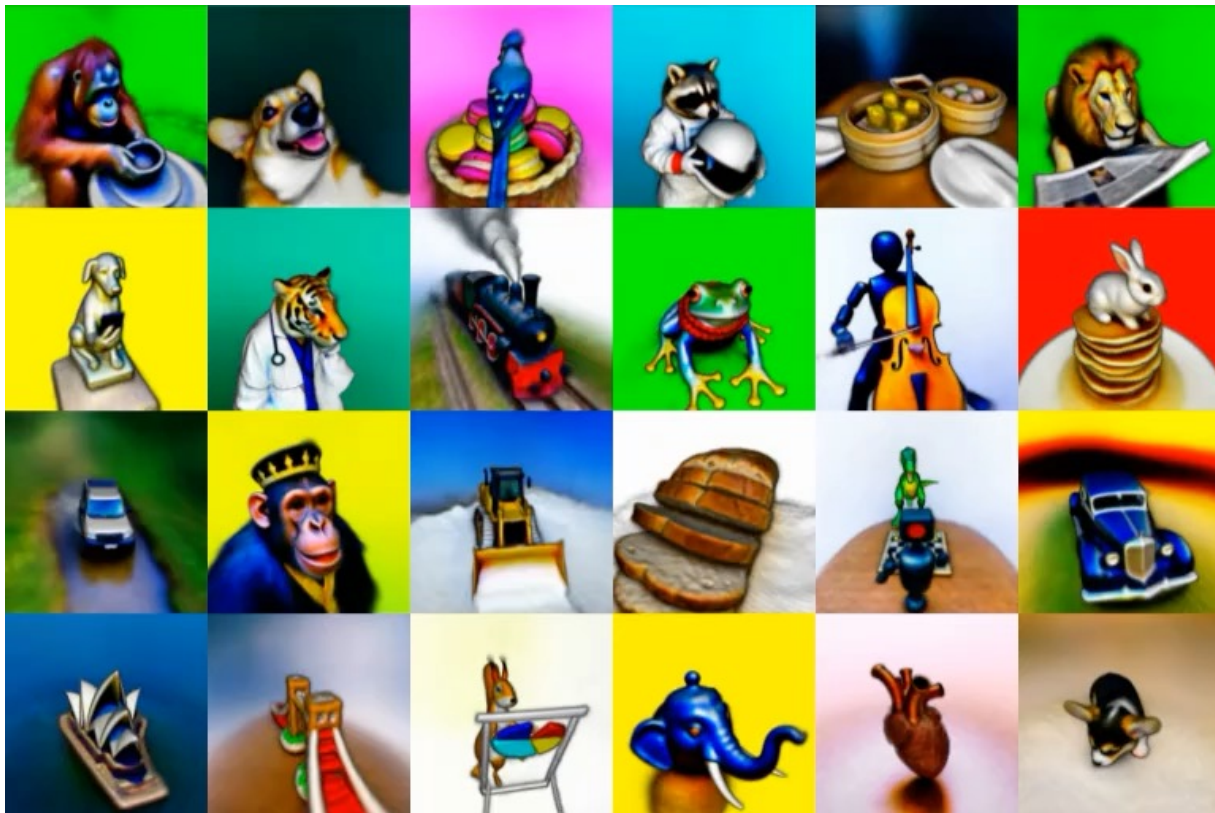
Matsuki et al, "Gaussian Splatting SLAM", CVPR 2024

Foundation Models for 3D Generation

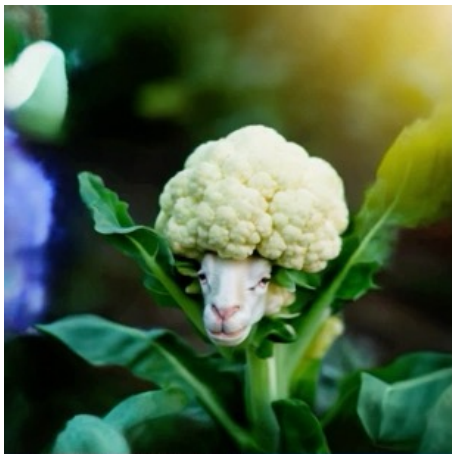
"a DSLR photo of a peacock on a surfboard"

DreamFusion

Automatic text-to-3D



DreamFusion: Text-to-3D using 2D Diffusion, Ben et al., arXiv 2022

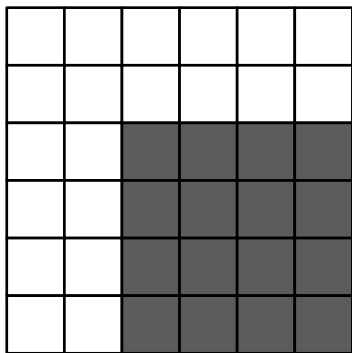


CAT3D: Create Anything in 3D with Multi-View Diffusion Models, Gao et al., arXiv 2024

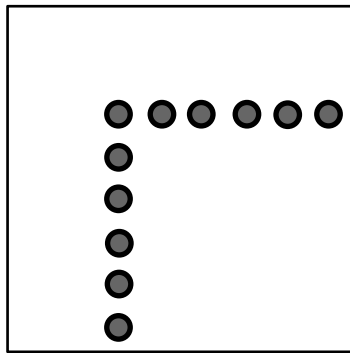
Summary: 3D Shape Representations



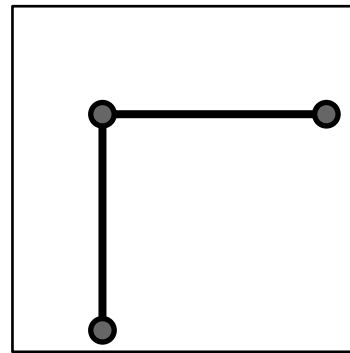
Depth
Map



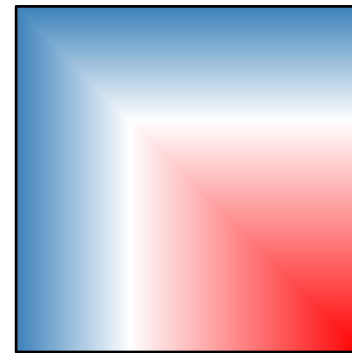
Voxel
Grid



Pointcloud



Mesh



Implicit
Surface